게임프로그래밍

디지털 시계를 활용한 프로그램

작성자 : 김재홍	2021963020

목차 페이지

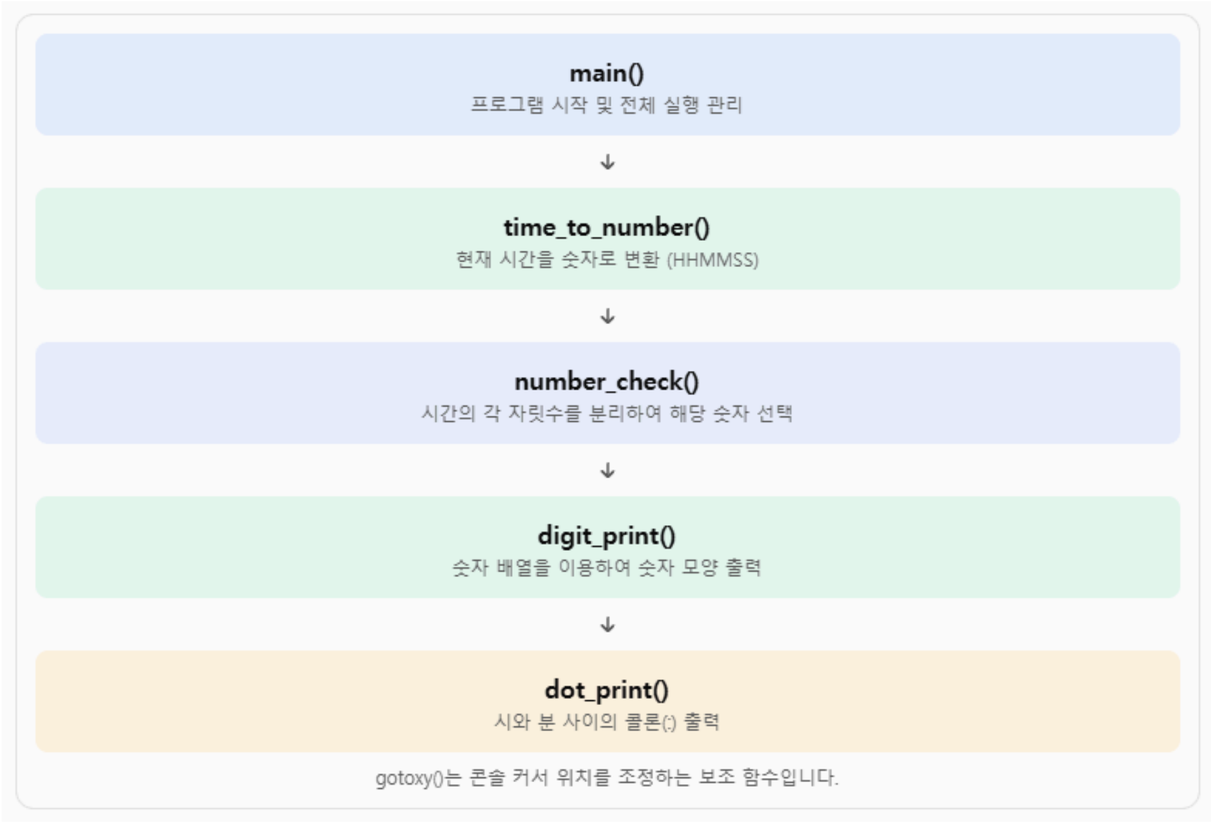
01 원본 소스 코드 분석

02 원본 소스 코드 수정

03 기능 추가(업그레이드)

04 점수

응용 10_1_1 코드 전체 구조



응용 10_1_1 코드

헤더 파일 및 함수 원형 선언

```c
#include <stdio.h>        //printf() 등 입출력 함수
#include <time.h>         //현재 시간 정보를 가져오는 함수
#include <stdlib.h>       //system() 함수
#include <conio.h>        //getch(), kbhit() 키보드 입력 함수
#include <windows.h>      //Windows 콘솔 커서 제어 기능

long time_to_number(void);        //현재 시간을 정수로 변환
void number_check(int, int);      //숫자 자릿수 판별
void digit_print(int*, int);      //숫자 모양 출력
void gotoxy(int x, int y);        //콘솔 커서 위치 이동
void dot_print(int hour);         //시간 구분점 출력
```

응용 10_1_1 코드

디지털 숫자 표현하는 배열

```
int zero[20] =
    {1,1,1,1,
     1,0,0,1,
     1,0,0,1,
     1,0,0,1,
     1,1,1,1};

int two[20]=
 {1,1,1,1,
  0,0,0,1,
  1,1,1,1,
  1,0,0,0,
  1,1,1,1};

int four[20]=
 {1,0,0,1,
  1,0,0,1,
  1,1,1,1,
  0,0,0,1,
  0,0,0,1};

int six[20]=
 {1,0,0,0,
  1,0,0,0,
  1,1,1,1,
  1,0,0,1,
  1,1,1,1};

int eight[20]=
 {1,1,1,1,
  1,0,0,1,
  1,1,1,1,
  1,0,0,1,
  1,1,1,1};

int one[20]=
 {0,0,1,0,
  0,0,1,0,
  0,0,1,0,
  0,0,1,0,
  0,0,1,0};

int three[20]=
 {1,1,1,1,
  0,0,0,1,
  1,1,1,1,
  0,0,0,1,
  1,1,1,1};

int five[20]=
 {1,1,1,1,
  1,0,0,0,
  1,1,1,1,
  0,0,0,1,
  1,1,1,1};

int seven[20]=
 {1,1,1,1,
  0,0,0,1,
  0,0,0,1,
  0,0,0,1,
  0,0,0,1};

int nine[20] =
    {1,1,1,1,
     1,0,0,1,
     1,1,1,1,
     0,0,0,1,
     0,0,0,1};
```

응용 10_1_1 코드

main() 함수 분석

```c
int main(void)
{
    int line;
    long time_number;
    printf("디지털 시계 프로그램\n\n");
    printf("아무 키나 누르면 현재 시간을 출력합니다.\n");
    getch();
    system("cls");
    while(!kbhit())
    {
        gotoxy(1,3);
        time_number=time_to_number();
        for(line=0;line<=4;line++)
        {
            number_check(time_number, line);
            printf("\n");
        }
        dot_print(time_number);
        printf("\n\n\n\n아무 키나 누르면 종료합니다.\n");
    }
    return 0;
}
```

프로그램 실행 및 반복 관리
반복문, 조건문

응용 10_1_1 코드

time_to_number() 함수 분석

```
long time_to_number(void)
{
    time_t current;
    struct tm *d;
    int hour, min, sec;
    current=time(NULL);
    d=localtime(&current);
    hour=d->tm_hour*10000;
    min=d->tm_min*100;
    sec=d->tm_sec;
    return hour+min+sec;
}
```

현재 시간을 숫자로 변환
시간 구조체, 포인터

응용 10_1_1 코드

number_check() 함수 분석

```
void number_check(int k, int i)
{
 if (k>=1)
  {
     number_check(k/10, i);
     switch(k%10)
     {
        case 0 : digit_print(zero, i); break;
        case 1 : digit_print(one, i); break;
        case 2 : digit_print(two, i); break;
        case 3 : digit_print(three, i); break;
        case 4 : digit_print(four, i); break;
        case 5 : digit_print(five, i); break;
        case 6 : digit_print(six, i); break;
        case 7 : digit_print(seven, i); break;
        case 8 : digit_print(eight, i); break;
        case 9 : digit_print(nine, i); break;
     }
  }
}
```

각 자릿수 판별
재귀 함수, 나머지 연산

응용 10_1_1 코드

digit_print() 함수 분석

```
140  void digit_print(int dim[], int check)
141  {
142
143   int i;
144   for(i=line*4;i<=line*4+3; i++)
145    if (dim[i]==1)
146       printf("■");
147    else
148       printf("  ");
149   printf("  ");
150  }
```

숫자 모양 출력
배열, 반복문

응용 10_1_1 코드

dot_print() 함수 분석

```
void dot_print(int hour)
{
    int i;
    if (hour<100000)
    {
        for(i=0;i<2;i++)
        {
            gotoxy(10+(i*27), 4);
            printf("■");
            gotoxy(10+(i*27), 6);
            printf("■");
        }
    }
    else
    {
        for(i=0;i<2;i++)
        {
            gotoxy(24+(i*26), 4);
            printf("■");
            gotoxy(24+(i*26), 6);
            printf("■");
        }
    }

}
```

시간 구분점 출력
조건문, 좌표 계산

응용 10_1_1 코드

gotoxy() 함수 분석

```
void gotoxy(int x, int y)
{
    COORD Pos = {x - 1, y - 1};
    SetConsoleCursorPosition(GetStdHandle(STD_OUTPUT_HANDLE), Pos);
}
```

콘솔 커서 이동

원본 코드 수정

```c
int main(void)
{
    int line;
    long time_number;
    printf("디지털 시계 프로그램\n\n");
    printf("아무 키나 누르면 현재 시간을 출력합니다.\n");
    getch();
    system("cls");
    while(!kbhit())
    {
        gotoxy(1,3);
        time_number=time_to_number();
        for(line=0;line<=4;line++)
        {
            number_check(time_number, line);
            printf("\n");
        }
        dot_print(time_number);
        printf("\n\n\n\n아무 키나 누르면 종료합니다.\n");
    }
    return 0;
}
```

```c
140  void digit_print(int dim[], int check)
141  {
142
143   int i;
144   for(i=line*4;i<=line*4+3; i++)
145    if (dim[i]==1)
146        printf("■");
147    else
148        printf("  ");
149   printf("  ");
150  }
```

원본 코드 수정

```
140  void digit_print(int dim[], int check)
141  {
142
143   int i;
144   for(i=line*4;i<=line*4+3; i++)
145    if (dim[i]==1)
146       printf("■");
147    else
148       printf("  ");
149   printf("  ");
150  }
```

```
void digit_print(int *dim, int check)
{
    int i;

    for (i = check * 4; i <= check * 4 + 3; i++)
    {
        if (dim[i] == 1)
            printf("■");
        else
            printf("  ");
    }

    printf("  ");
}
```

기능 추가(1) 모드 선택

```c
// 메인 함수
int main(void)
{
    int line;
    int mode;
    int ch;
    long time_number;

    // 모드 선택 화면
    printf("===== 디지털 프로그램 =====\n\n");
    printf("1. 현재 시간\n");
    printf("2. 추후 추가 예정\n");
    printf("3. 추후 추가 예정\n");
    printf("4. 추후 추가 예정\n\n");

    printf("원하는 모드의 숫자를 입력하세요: ");

    // 숫자 입력
    if (scanf("%d", &mode) != 1)
    {
        printf("잘못된 입력입니다.\n");
        return 1;
    }

    // 입력 버퍼에 남아 있는 Enter 키 제거
    while ((ch = getchar()) != '\n' && ch != EOF)
    {
    }

    // 모드 선택
    if (mode == 1)
    {
        system("cls");

        printf("현재 시간 모드를 시작합니다.\n");
        printf("아무 키나 누르면 종료합니다.\n");

        getch();
        system("cls");

        // 기존 디지털 시계 기능
        while (!kbhit())
        {
            gotoxy(1, 3);

            time_number = time_to_number();

            for (line = 0; line <= 4; line++)
            {
                number_check((int)time_number, line);
                printf("\n");
            }

            dot_print((int)time_number);

            printf("\n\n\n\n아무 키나 누르면 종료합니다.\n");
        }
    }
    else
    {
        printf("\n아직 구현되지 않은 모드입니다.\n");
        printf("프로그램을 종료합니다.\n");
    }

    return 0;
```

기능 추가(1) 모드 선택

```
===== 디지털 프로그램 =====

1. 현재 시간
2. 추후 추가 예정
3. 추후 추가 예정
4. 추후 추가 예정

원하는 모드의 숫자를 입력하세요:
```

```
1. 현재 시간
2. 추후 추가 예정
3. 추후 추가 예정
4. 추후 추가 예정

원하는 모드의 숫자를 입력하세요: 1
현재 시간 모드를 시작합니다.
아무 키나 누르면 종료합니다.
```

```
원하는 모드의 숫자를 입력하세요: 3

아직 구현되지 않은 모드입니다.
프로그램을 종료합니다.
```

기능 추가(2) 뉴욕 시간

```c
/* 뉴욕 시간을 표준 C 함수만 사용해 HHMMSS 형태로 반환합니다. */
long new_york_time_to_number(void)
{
    time_t current_utc = time(NULL);
    struct tm *utc_time;
    struct tm utc_copy;
    int leap_year;
    int march_first_weekday;
    int november_first_weekday;
    int second_sunday_march;
    int first_sunday_november;
    int is_daylight_saving;
    int utc_day;
    int utc_hour;
    int offset;
```

```c
    if (current_utc == (time_t)-1)
        return 0;

    utc_time = gmtime(&current_utc);

    if (utc_time == NULL)
        return 0;

    utc_copy = *utc_time;

    /* 윤년 여부를 확인합니다. */
    leap_year =
        (utc_copy.tm_year % 4 == 0 &&
        (utc_copy.tm_year % 100 != 0 ||
         (utc_copy.tm_year + 1900) % 400 == 0));
```

기능 추가(2) 뉴욕 시간

```
/*
   미국 서머타임 시작:
   3월 둘째 일요일 오전 2시(뉴욕 현지 시간)
   이는 UTC 기준 오전 7시입니다.

   미국 서머타임 종료:
   11월 첫째 일요일 오전 2시(뉴욕 현지 시간)
   이는 UTC 기준 오전 6시입니다.
*/
march_first_weekday =
    (utc_copy.tm_wday - (utc_copy.tm_yday -
    (31 + 28 + leap_year)) % 7 + 7) % 7;

november_first_weekday =
    (utc_copy.tm_wday - (utc_copy.tm_yday -
    (304 + leap_year)) % 7 + 7) % 7;

second_sunday_march =
    31 + 28 + leap_year +
    (7 - march_first_weekday) % 7 + 7;

first_sunday_november =
    304 + leap_year +
    (7 - november_first_weekday) % 7;

utc_day = utc_copy.tm_yday;
utc_hour = utc_copy.tm_hour;

is_daylight_saving = 0;
```

```
if ((utc_day > second_sunday_march ||
     (utc_day == second_sunday_march && utc_hour >= 7)) &&
    (utc_day < first_sunday_november ||
     (utc_day == first_sunday_november && utc_hour < 6)))
{
    is_daylight_saving = 1;
}

/* 뉴욕은 서머타임 UTC-4, 표준시 UTC-5입니다. */
if (is_daylight_saving)
    offset = -4;
else
    offset = -5;

/* UTC 시각에 시차를 적용한 뒤 gmtime()으로 시각을 구합니다. */
current_utc += offset * 60 * 60;
utc_time = gmtime(&current_utc);

if (utc_time == NULL)
    return 0;

return utc_time->tm_hour * 10000L
     + utc_time->tm_min * 100L
     + utc_time->tm_sec;
```

```
===== 디지털 시계 프로그램 =====

1. 현재 컴퓨터 시간
2. 뉴욕 시간

원하는 모드를 입력하세요: 2
현재 컴퓨터 시간

###### ###### ##  ###### ###### ######
#  ## ##  ## ## ##  ##  ## ##  ## ##  ##
#  ## ##  ##    ##  ###### ##  ## ##  ##
#  ## ##  ## ## ##  ##  ## ##  ## ##  ##
###### ###### ##  ###### ###### ######

아무 키나 누르면 메뉴로 돌아갑니다.

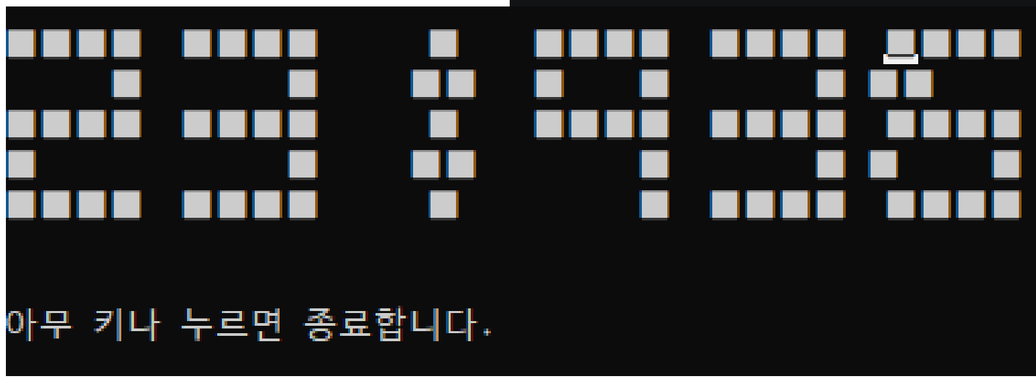
뉴욕 시간

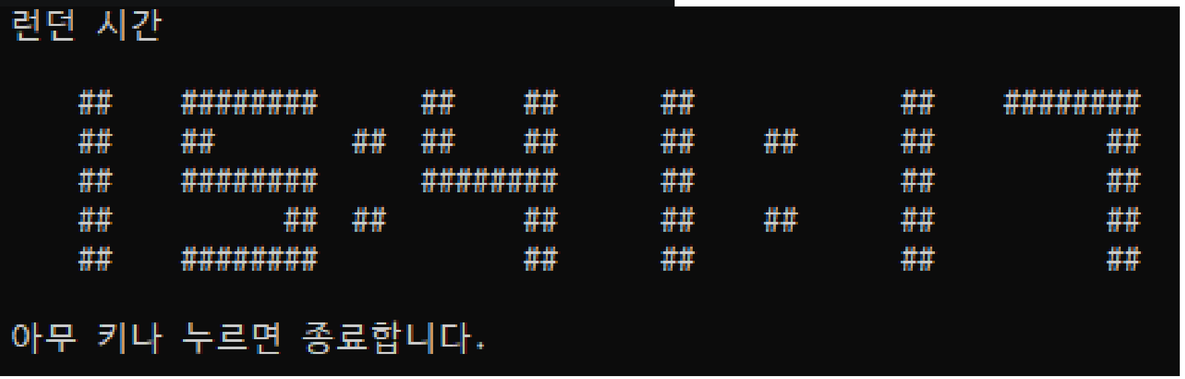
   ##  ###### ##  ###### ##  ## ######
   ## ##  ## ## ##  ##  ## ## ##  ##
   ## ##  ##    ##  ###### ##  ##
   ## ##  ## ## ##  ##  ## ##  ##
   ## ###### ##  ###### ##  ##

아무 키나 누르면 메뉴로 돌아갑니다.
```

기능 추가(3) 디지털 표기 변경

```
if (number[row * 4 + col] == 1)
    printf("##");
else
    printf("  ");
```



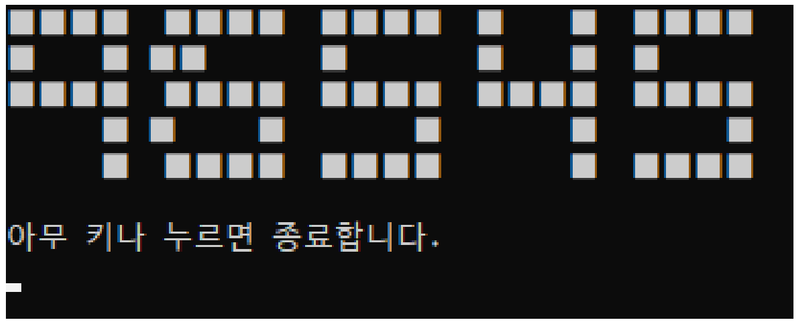


기능 추가(4) 숫자 출력 함수 변경

```
void print_digital_time(long time_number)
{
    int hour   = (int)(time_number / 10000);
    int minute = (int)((time_number / 100) % 100);
    int second = (int)(time_number % 100);

    int digits[6];
```

새 출력 방식은 시·분·초를 각각 두 자리로 표시해 앞의 0도 출력





기능 추가(5) 폰트 색깔 변경

```c
/* 콘솔 글자 색상을 설정합니다. */
void set_text_color(WORD color)
{
    HANDLE console = GetStdHandle(STD_OUTPUT_HANDLE);
    SetConsoleTextAttribute(console, color);
}
```

```c
HANDLE console = GetStdHandle(STD_OUTPUT_HANDLE);
CONSOLE_SCREEN_BUFFER_INFO info;
WORD original_color = FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_BLUE;

if (GetConsoleScreenBufferInfo(console, &info))
    original_color = info.wAttributes;

digits[0] = hour / 10;
digits[1] = hour % 10;
digits[2] = minute / 10;
digits[3] = minute % 10;
digits[4] = second / 10;
digits[5] = second % 10;

/* 초록색과 파란색을 켜서 하늘색 계열로 설정합니다. */
set_text_color(FOREGROUND_GREEN | FOREGROUND_BLUE | FOREGROUND_INTENSITY);
```

```c
/* 제목과 안내 문구는 원래 콘솔 색상으로 복원합니다. */
set_text_color(original_color);
```

뉴욕 시간

```
  ##    ########     ########  ########         ##    ##
  ##    ##    ##  ##  ##              ##  ##     ##    ##
  ##    ##    ##     ########  ########         ##    ########
  ##    ##    ##  ##       ##  ##       ##     ##    ##    ##
  ##    ########     ########  ########         ##    ########
```

기능 추가(6) 분 단위로 효과음 발생

```
/* 분이 바뀔 때 짧은 효과음을 냅니다. */
void play_minute_sound(void)
{
    Beep(880, 150);
}
```

기능 추가(7) 모드로 복귀

```
        printf("\n아무 키나 누르면 메뉴로 돌아갑니다.\n");
        Sleep(200);
    }

    /* 모드 종료를 위해 입력된 키를 읽고, 메뉴로 돌아갑니다. */
    _getch();
}
```

기능 추가(8) 0으로 프로그램 종료

```
if (mode == 0)
{
    printf("프로그램을 종료합니다.\n");
    break;
}
```

기능 추가(9) 키 입력 횟수 게임

```c
/* 확장 키도 한 번의 입력으로 처리합니다. */
int read_key_press(void)
{
    int key = _getch();

    /* 방향키, 기능키 등 일부 특수 키는 추가 코드가 전달됩니다. */
    if (key == 0 || key == 224)
        _getch();

    return key;
}
```

```c
/* 10초 동안 키 입력 횟수를 세는 게임입니다. */
void play_key_count_game(void)
{
    int key_count = 0;
    int elapsed_seconds;
    int remaining_seconds;
    int previous_remaining = -1;
    clock_t start_time;
    clock_t current_time;

    system("cls");

    printf("===== 키 입력 횟수 게임 =====\n\n");
    printf("아무 키나 한 번 누르면 게임이 시작됩니다.\n");
    printf("시작을 알리는 키 입력은 횟수에 포함하지 않습니다.\n");
```

기능 추가(9) 키 입력 횟수 게임

```
/* 첫 번째 키 입력을 시작 신호로 사용합니다. */
read_key_press();
start_time = clock();

/* 제한 시간 10초 동안 키 입력을 받습니다. */
while (1)
{
    current_time = clock();
    elapsed_seconds =
        (int)((double)(current_time - start_time) / CLOCKS_PER_SEC);

    if (elapsed_seconds >= 10)
        break;

    remaining_seconds = 10 - elapsed_seconds;

    /* 남은 시간이 바뀔 때만 화면을 다시 출력합니다. */
    if (remaining_seconds != previous_remaining)
    {
        system("cls");

        printf("===== 키 입력 횟수 게임 =====\n\n");
        printf("남은 시간\n\n");
        print_digital_time((long)remaining_seconds);
        printf("\n현재 입력 횟수: %d\n", key_count);

        previous_remaining = remaining_seconds;
    }

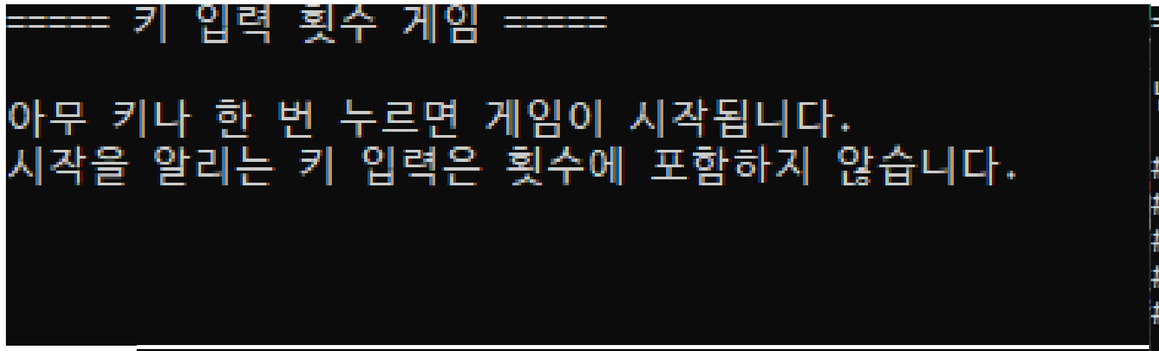
    /* 입력된 키가 있으면 한 번의 입력으로 계산합니다. */
    if (_kbhit())
    {
        read_key_press();
        key_count++;
    }
}

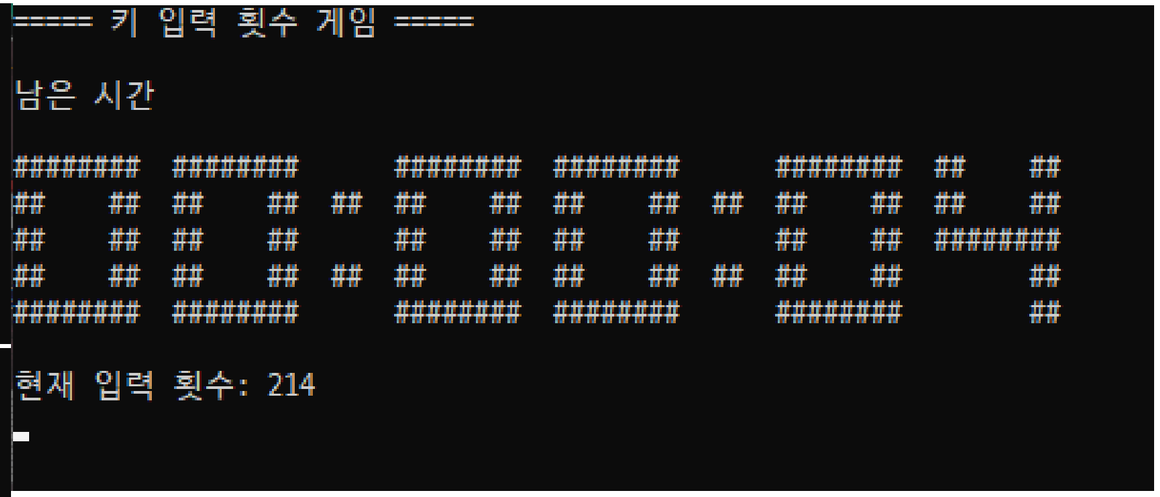
system("cls");

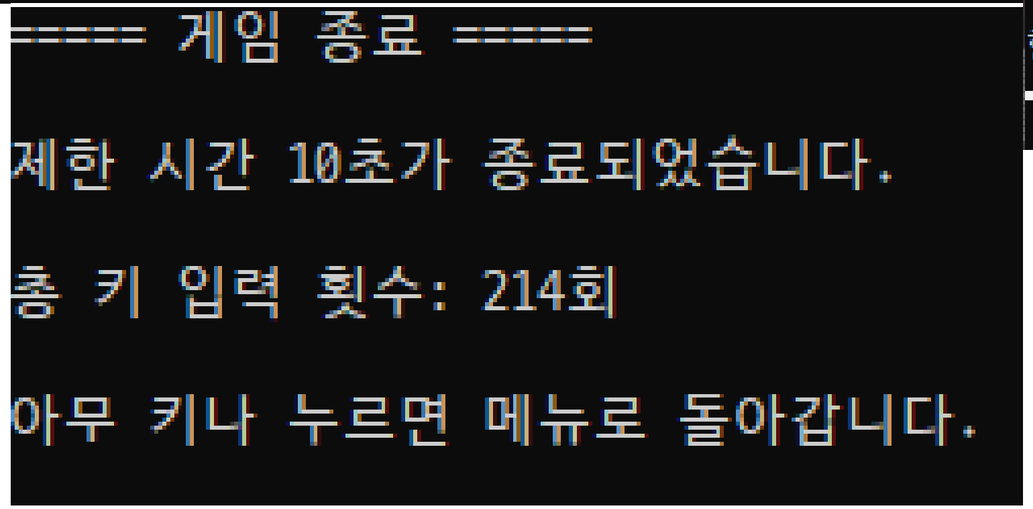
printf("===== 게임 종료 =====\n\n");
printf("제한 시간 10초가 종료되었습니다.\n\n");
printf("총 키 입력 횟수: %d회\n\n", key_count);
printf("아무 키나 누르면 메뉴로 돌아갑니다.\n");

_getch();
```

기능 추가(9) 키 입력 횟수 게임







기능 추가(10) 키 입력 횟수 게임-키 입력 무시

```
/* 게임 종료 후 3초 동안 입력을 버려 점수 화면이 유지되도록 합니다. */
result_start_time = clock();

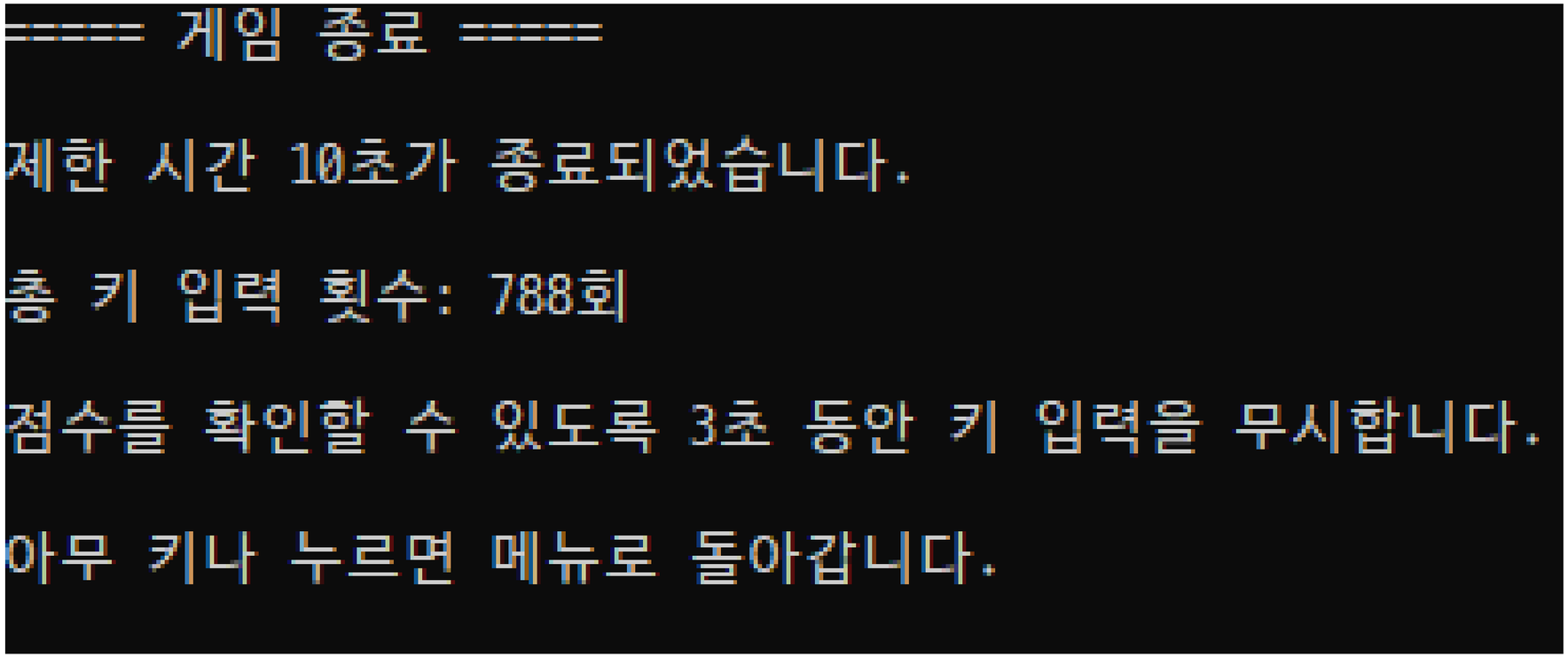
while ((double)(clock() - result_start_time) / CLOCKS_PER_SEC < 3.0)
{
    /* 10초 게임 중 남아 있는 입력과 3초 동안의 입력을 모두 비웁니다. */
    while (_kbhit())
    {
        read_key_press();
    }
}

/* 3초가 지난 뒤부터 새로운 키 입력을 받아 메뉴로 돌아갑니다. */
printf("\n아무 키나 누르면 메뉴로 돌아갑니다.\n");

while (_kbhit())
{
    read_key_press();
}

_getch();
```

기능 추가(10) 키 입력 횟수 게임-키 입력 무시



기능 추가(11) 키 입력 횟수 게임-최고기록 저장

```c
/* 텍스트 파일에서 최고 기록을 읽어옵니다. */
int load_best_score(void)
{
    FILE *record_file;
    int best_score = 0;

    record_file = fopen("record.txt", "r");

    /* 기록 파일이 없으면 최초 실행으로 간주하고 0회를 반환합니다. */
    if (record_file == NULL)
        return 0;

    /* 파일에 저장된 숫자를 최고 기록으로 읽습니다. */
    if (fscanf(record_file, "%d", &best_score) != 1 || best_score < 0)
        best_score = 0;

    fclose(record_file);

    return best_score;
}
```

```c
/* 새로운 최고 기록을 텍스트 파일에 저장합니다. */
int save_best_score(int score)
{
    FILE *record_file;

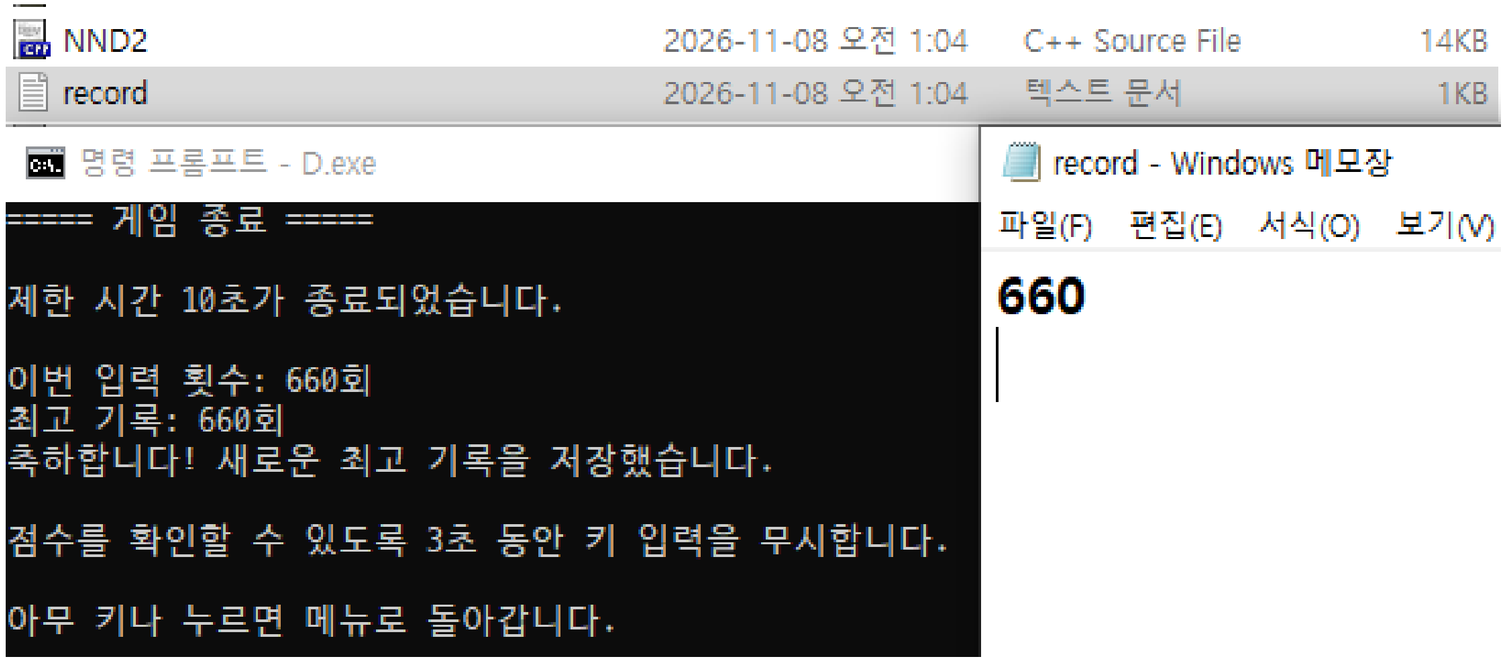
    record_file = fopen("record.txt", "w");

    if (record_file == NULL)
        return 0;

    fprintf(record_file, "%d\n", score);
    fclose(record_file);

    return 1;
}
```

기능 추가(11) 키 입력 횟수 게임-최고기록 저장



기능 추가(12) 8자리 이진수 표기

```c
/* 0부터 255까지의 정수를 8자리 디지털 이진수로 표시합니다. */
void print_binary_number(int value)
{
    int bits[8];
    int i;
    int row;

    /* 8비트의 각 자리를 왼쪽부터 계산합니다. */
    for (i = 0; i < 8; i++)
    {
        bits[i] = (value >> (7 - i)) & 1;
    }

    /* 기존 디지털 숫자 출력 함수를 이용해 0과 1을 표시합니다. */
    for (row = 0; row < 5; row++)
    {
        for (i = 0; i < 8; i++)
        {
            print_digit_row(bits[i], row);

            if (i < 7)
                printf("  ");
        }

        printf("\n");
    }
}
```

```c
/* 입력한 정수를 검증한 뒤 8자리 이진수로 출력하는 모드입니다. */
void play_binary_display_mode(void)
{
    int value;
    int ch;

    system("cls");

    printf("===== 8자리 이진수 표기 =====\n\n");
    printf("0부터 255까지의 정수를 입력하세요: ");

    if (scanf("%d", &value) != 1)
    {
        printf("잘못된 입력입니다. 정수만 입력할 수 있습니다.\n");

        while ((ch = getchar()) != '\n' && ch != EOF)
        {
        }

        printf("\n아무 키나 누르면 메뉴로 돌아갑니다.\n");
        _getch();
        return;
    }

    while ((ch = getchar()) != '\n' && ch != EOF)
    {
    }

    if (value < 0 || value > 255)
    {
        printf("입력 범위를 벗어났습니다. 0부터 255까지만 입력할 수 있습니다.\n");
    }
    else
    {
        printf("\n입력한 정수: %d\n", value);
        printf("8자리 이진수: \n\n");
        print_binary_number(value);
    }

    printf("\n아무 키나 누르면 메뉴로 돌아갑니다.\n");
    _getch();
```

기능 추가(12) 8자리 이진수 표기

```
===== 8자리 이진수 표기 =====

0부터 255까지의 정수를 입력하세요: 60

입력한 정수: 60
8자리 이진수:

########  ########      ##        ##        ##        ##    ########  ########
##    ##  ##    ##      ##        ##        ##        ##    ##    ##  ##    ##
##    ##  ##    ##      ##        ##        ##        ##    ##    ##  ##    ##
##    ##  ##    ##      ##        ##        ##        ##    ##    ##  ##    ##
########  ########      ##        ##        ##        ##    ########  ########

아무 키나 누르면 메뉴로 돌아갑니다.
```

```
===== 8자리 이진수 표기 =====

0부터 255까지의 정수를 입력하세요: 256
입력 범위를 벗어났습니다. 0부터 255까지만 입력할 수 있습니다.

아무 키나 누르면 메뉴로 돌아갑니다.
```

기능 추가(13) 8자리 이진수 맞히기

```c
/* 4번 모드에서 실행할 하위 메뉴입니다. */
void play_binary_mode_menu(void)
{
    int choice;
    int ch;

    while (1)
    {
        system("cls");

        printf("===== 이진수 모드 =====\n\n");
        printf("1. 8자리 이진수 표기\n");
        printf("2. 이진수 보고 정수 맞히기 게임\n");
        printf("0. 이전 메뉴로 돌아가기\n\n");
        printf("원하는 기능을 선택하세요: ");

        if (scanf("%d", &choice) != 1)
        {
            printf("잘못된 입력입니다. 숫자를 입력하세요.\n");

            while ((ch = getchar()) != '\n' && ch != EOF)
            {
            }

            wait_seconds(1.0);
            continue;
        }

        while ((ch = getchar()) != '\n' && ch != EOF)
        {
        }

        if (choice == 0)
            return;
        else if (choice == 1)
            play_binary_display_mode();
        else if (choice == 2)
            play_binary_guess_game();
        else
        {
            printf("잘못된 메뉴 선택입니다.\n");
            wait_seconds(1.0);
        }
    }
}
```

```c
/* 디지털 이진수를 보고 원래의 정수를 맞히는 게임입니다. */
void play_binary_guess_game(void)
{
    int answer;
    int guess;
    int attempts = 0;
    int ch;

    /* 0부터 255 사이의 정수를 무작위로 선택합니다. */
    answer = rand() % 256;

    system("cls");

    printf("===== 이진수 보고 정수 맞히기 =====\n\n");
    printf("아래에 표시된 8자리 이진수가 나타내는 정수를 맞혀보세요.\n");
    printf("정답은 0부터 255 사이의 정수입니다.\n\n");

    print_binary_number(answer);

    printf("\n");

    /* 정답을 맞힐 때까지 입력을 반복합니다. */
    while (1)
    {
        printf("정수 입력: ");

        if (scanf("%d", &guess) != 1)
        {
            printf("잘못된 입력입니다. 정수를 입력하세요.\n");

            while ((ch = getchar()) != '\n' && ch != EOF)
            {
            }

            continue;
        }

        while ((ch = getchar()) != '\n' && ch != EOF)
        {
        }

        if (guess < 0 || guess > 255)
        {
            printf("0부터 255 사이의 정수만 입력할 수 있습니다.\n\n");
            continue;
        }

        attempts++;

        if (guess == answer)
        {
            printf("\n정답입니다!\n");
            printf("정답: %d\n", answer);
            printf("시도 횟수: %d회\n", attempts);
            break;
        }
        else if (guess < answer)
        {
            printf("틀렸습니다. 정답은 입력한 값보다 큰 수입니다.\n\n");
        }
        else
        {
            printf("틀렸습니다. 정답은 입력한 값보다 작은 수입니다.\n\n");
        }
    }

    printf("\n아무 키나 누르면 이전 메뉴로 돌아갑니다.\n");
    _getch();
```

기능 추가(13) 8자리 이진수 맞히기

```
===== 이진수 모드 =====

1. 8자리 이진수 표기
2. 이진수 보고 정수 맞히기 게임
0. 이전 메뉴로 돌아가기

원하는 기능을 선택하세요: 2
```

```
===== 이진수 보고 정수 맞히기 =====

아래에 표시된 8자리 이진수가 나타내는 정수를 맞혀보세요.
정답은 0부터 255 사이의 정수입니다.

   ##       ##       ##       ##    ########  ########  ########  ########
   ##       ##       ##       ##    ##    ##  ##    ##  ##    ##  ##    ##
   ##       ##       ##       ##    ##    ##  ##    ##  ##    ##  ##    ##
   ##       ##       ##       ##    ##    ##  ##    ##  ##    ##  ##    ##
   ##       ##       ##       ##    ########  ########  ########  ########

정수 입력: 240

정답입니다!
정답: 240
시도 횟수: 1회

아무 키나 누르면 이전 메뉴로 돌아갑니다.
```

```
===== 이진수 보고 정수 맞히기 =====

아래에 표시된 8자리 이진수가 나타내는 정수를 맞혀보세요.
정답은 0부터 255 사이의 정수입니다.

########  ########  ########      ##    ########  ########  ########  ########
##    ##  ##    ##  ##    ##      ##    ##    ##  ##    ##  ##    ##  ##    ##
##    ##  ##    ##  ##    ##      ##    ##    ##  ##    ##  ##    ##  ##    ##
##    ##  ##    ##  ##    ##      ##    ##    ##  ##    ##  ##    ##  ##    ##
########  ########  ########      ##    ########  ########  ########  ########

정수 입력: 17
틀렸습니다. 정답은 입력한 값보다 작은 수입니다.

정수 입력: 16

정답입니다!
정답: 16
시도 횟수: 2회

아무 키나 누르면 이전 메뉴로 돌아갑니다.
```

기능 추가(14) 8자리 이진수 2의 보수

```c
/* 입력한 정수의 8비트 2의 보수를 디지털 숫자로 표시합니다. */
void play_binary_twos_complement_mode(void)
{
    int value;
    int twos_complement;
    int ch;

    system("cls");

    printf("===== 8자리 이진수 2의 보수 표기 =====\n\n");
    printf("0부터 255까지의 정수를 입력하세요: ");

    if (scanf("%d", &value) != 1)
    {
        printf("잘못된 입력입니다. 정수만 입력할 수 있습니다.\n");

        while ((ch = getchar()) != '\n' && ch != EOF)
        {
        }

        printf("\n아무 키나 누르면 이전 메뉴로 돌아갑니다.\n");
        _getch();
        return;
    }
    while ((ch = getchar()) != '\n' && ch != EOF)
    {
    }

    if (value < 0 || value > 255)
    {
        printf("입력 범위를 벗어났습니다. 0부터 255까지만 입력할 수 있습니다.\n");
    }
    else
    {
        /* 8비트 범위에서 비트를 반전하고 1을 더합니다. */
        twos_complement = (~value + 1) & 0xFF;

        printf("\n입력한 정수: %d\n", value);
        printf("원래의 8자리 이진수:\n\n");
        print_binary_number(value);

        printf("\n2의 보수 결과: %d\n", twos_complement);
        printf("2의 보수 8자리 이진수:\n\n");
        print_binary_number(twos_complement);
    }

    printf("\n아무 키나 누르면 이전 메뉴로 돌아갑니다.\n");
    _getch();
```

기능 추가(14) 8자리 이진수 2의 보수

```
===== 이진수 모드 =====

1. 8자리 이진수 표기
2. 이진수 보고 정수 맞히기 게임
3. 8자리 이진수 2의 보수 표기
0. 이전 메뉴로 돌아가기

원하는 기능을 선택하세요: 3
```

```
===== 8자리 이진수 2의 보수 표기 =====

0부터 255까지의 정수를 입력하세요: 42

입력한 정수: 42
원래의 8자리 이진수:

########  ########    ##    ########    ##    ########    ##    ########
##    ##  ##    ##    ##    ##    ##    ##    ##    ##    ##    ##    ##
##    ##  ##    ##    ##    ##    ##    ##    ##    ##    ##    ##    ##
##    ##  ##    ##    ##    ##    ##    ##    ##    ##    ##    ##    ##
########  ########    ##    ########    ##    ########    ##    ########

2의 보수 결과: 214
2의 보수 8자리 이진수:

   ##        ##    ########    ##    ########    ##        ##    ########
   ##        ##    ##    ##    ##    ##    ##    ##        ##    ##    ##
   ##        ##    ##    ##    ##    ##    ##    ##        ##    ##    ##
   ##        ##    ##    ##    ##    ##    ##    ##        ##    ##    ##
   ##        ##    ########    ##    ########    ##        ##    ########

아무 키나 누르면 이전 메뉴로 돌아갑니다.
```

기능 추가(15) 8자리 이진수 시프트

```
while ((ch = getchar()) != '\n' && ch != EOF)
{
}

if (operator_choice == 1)
{
    /* 8비트 범위를 벗어난 비트는 버립니다. */
    result = (value << 1) & 0xFF;
}
else if (operator_choice == 2)
{
    /* 입력값이 0 이상이므로 오른쪽 시프트 결과는 0으로 채워집니다. */
    result = value >> 1;
}
else if (operator_choice == 3)
{
    /* unsigned 형식으로 변환하여 논리적 오른쪽 시프트를 수행합니다. */
    result = (int)((unsigned int)value >> 1);
}
else
{
    printf("잘못된 연산자 선택입니다. 1부터 3까지 입력하세요.\n");
    printf("\n아무 키나 누르면 이전 메뉴로 돌아갑니다.\n");
    _getch();
    return;
}
```

```
system("cls");

printf("===== 8자리 이진수 시프트 연산 결과 =====\n\n");
printf("입력한 정수: %d\n", value);
printf("연산 전 8자리 이진수:\n\n");
print_binary_number(value);

printf("\n선택한 연산자: ");

if (operator_choice == 1)
    printf("<< (왼쪽 시프트)\n");
else if (operator_choice == 2)
    printf(">> (오른쪽 시프트)\n");
else
    printf(">>> (논리적 오른쪽 시프트)\n");

printf("이동 횟수: 1비트\n");
printf("\n연산 결과 정수: %d\n", result);
printf("연산 결과 8자리 이진수:\n\n");
print_binary_number(result);

printf("\n아무 키나 누르면 이전 메뉴로 돌아갑니다.\n");
_getch();
```

기능 추가(15) 8자리 이진수 시프트

```
===== 이진수 모드 =====

1. 8자리 이진수 표기
2. 이진수 보고 정수 맞히기 게임
3. 8자리 이진수 2의 보수 표기
4. 8자리 이진수 시프트 연산
0. 이전 메뉴로 돌아가기

원하는 기능을 선택하세요: 4
```

```
===== 8자리 이진수 시프트 연산 결과 =====

입력한 정수: 42
연산 전 8자리 이진수:

########  ########    ##    ########    ##    ########    ##    ########
##    ##  ##    ##    ##    ##    ##    ##    ##    ##    ##    ##    ##
##    ##  ##    ##    ##    ##    ##    ##    ##    ##    ##    ##    ##
##    ##  ##    ##    ##    ##    ##    ##    ##    ##    ##    ##    ##
########  ########    ##    ########    ##    ########    ##    ########

선택한 연산자: >> (오른쪽 시프트)
이동 횟수: 1비트

연산 결과 정수: 21
연산 결과 8자리 이진수:

########  ########  ########    ##    ########    ##    ########    ##
##    ##  ##    ##  ##    ##    ##    ##    ##    ##    ##    ##    ##
##    ##  ##    ##  ##    ##    ##    ##    ##    ##    ##    ##    ##
##    ##  ##    ##  ##    ##    ##    ##    ##    ##    ##    ##    ##
########  ########  ########    ##    ########    ##    ########    ##

아무 키나 누르면 이전 메뉴로 돌아갑니다.
```

기능 추가(16) 부호 있는 8비트 정수

```c
/* 부호 없는 8비트 또는 부호 있는 8비트 정수를 선택해 입력받습니다. */
int read_signed_binary_value(const char *title, int *value, int *is_signed)
{
    int number_choice;
    int input_value;
    int ch;

    system("cls");
    printf("===== %s =====\n\n", title);
    printf("표현 방식을 선택하세요.\n");
    printf("1. 양수 / 부호 없는 정수 (0 ~ 255)\n");
    printf("2. 음수 포함 / 부호 있는 정수 (-128 ~ 127)\n\n");
    printf("선택: ");

    if (scanf("%d", &number_choice) != 1)
    {
        printf("잘못된 입력입니다.\n");
        while ((ch = getchar()) != '\n' && ch != EOF) {}
        printf("\n아무 키나 누르면 이전 메뉴로 돌아갑니다.\n");
        _getch();
        return 0;
    }
    while ((ch = getchar()) != '\n' && ch != EOF) {}

    if (number_choice != 1 && number_choice != 2)
    {
        printf("1 또는 2만 선택할 수 있습니다.\n");
        printf("\n아무 키나 누르면 이전 메뉴로 돌아갑니다.\n");
        _getch();
        return 0;
    }

    if (number_choice == 1)
    {
        *is_signed = 0;
        printf("\n양수(부호 없는 정수)를 입력하세요 (0~255): ");
    }
    else
    {
        *is_signed = 1;
        printf("\n부호 있는 정수를 입력하세요 (-128~127): ");
    }

    if (scanf("%d", &input_value) != 1)
    {
        printf("잘못된 입력입니다. 정수를 입력하세요.\n");
        while ((ch = getchar()) != '\n' && ch != EOF) {}
        printf("\n아무 키나 누르면 이전 메뉴로 돌아갑니다.\n");
        _getch();
        return 0;
    }
    while ((ch = getchar()) != '\n' && ch != EOF) {}

    if (!*is_signed)
    {
        if (input_value < 0 || input_value > 255)
        {
            printf("부호 없는 정수는 0부터 255까지만 입력할 수 있습니다.\n");
            printf("\n아무 키나 누르면 이전 메뉴로 돌아갑니다.\n");
            _getch();
            return 0;
        }
    }
    else
    {
        if (input_value < -128 || input_value > 127)
        {
            printf("부호 있는 정수는 -128부터 127까지만 입력할 수 있습니다.\n");
            printf("\n아무 키나 누르면 이전 메뉴로 돌아갑니다.\n");
            _getch();
            return 0;
        }
    }

    *value = input_value;
    return 1;
```

기능 추가(16) 부호 있는 8비트 정수

```
else if (operator_choice == 2)
{
    /* 부호 있는 음수만 산술적 오른쪽 시프트로 부호 비트를 유지합니다
       부호 없는 정수는 왼쪽 빈자리를 0으로 채웁니다. */
    result_bits = bits >> 1;
    if (is_signed && (bits & 0x80))
        result_bits |= 0x80;
}
/* 입력 표현 방식에 맞춰 결과 비트 패턴을 정수로 해석합니다. */
if (is_signed)
    result_value = (result_bits >= 128) ? result_bits - 256 : result_bits;
else
    result_value = result_bits;
```

```
if (number_choice == 1)
{
    answer = rand() % 256;
}
else if (number_choice == 2)
{
    answer = (rand() % 256) - 128;
}
```

기능 추가(16) 부호 있는 8비트 정수

```
===== 8자리 이진수 표기 =====

표현 방식을 선택하세요.
1. 양수 / 부호 없는 정수 (0 ~ 255)
2. 음수 포함 / 부호 있는 정수 (-128 ~ 127)

선택:
```

```
===== 8자리 이진수 표기 =====

입력한 정수: 255
표현 방식: 부호 없는 8비트 정수
8비트 이진수 표현:

   ##       ##       ##       ##       ##       ##       ##       ##
   ##       ##       ##       ##       ##       ##       ##       ##
   ##       ##       ##       ##       ##       ##       ##       ##
   ##       ##       ##       ##       ##       ##       ##       ##
   ##       ##       ##       ##       ##       ##       ##       ##

아무 키나 누르면 이전 메뉴로 돌아갑니다.
```

```
===== 8자리 이진수 표기 =====

입력한 정수: -128
표현 방식: 부호 있는 8비트 정수
8비트 이진수 표현:

   ##    ########  ########  ########  ########  ########  ########  ########
   ##    ##    ##  ##    ##  ##    ##  ##    ##  ##    ##  ##    ##  ##    ##
   ##    ##    ##  ##    ##  ##    ##  ##    ##  ##    ##  ##    ##  ##    ##
   ##    ##    ##  ##    ##  ##    ##  ##    ##  ##    ##  ##    ##  ##    ##
   ##    ########  ########  ########  ########  ########  ########  ########

아무 키나 누르면 이전 메뉴로 돌아갑니다.
```

기능 추가(17) 스톱워치

```c
/* 디지털 숫자로 경과 시간을 표시하는 스톱워치입니다. */
void play_stopwatch_mode(void)
{
    time_t start_time;
    time_t current_time;
    double elapsed_seconds;
    long display_time;
    int key;
    int stopped = 0;

    system("cls");

    printf("===== 디지털 스톱워치 =====\n\n");
    printf("아무 키나 누르면 스톱워치가 시작됩니다.\n");
    printf("시간을 멈추려면 j 키를 누르세요.\n");
```

```c
    /* 시작을 알리는 첫 번째 키 입력은 시간 측정에 포함하지 않습니다. */
    read_key_press();
    start_time = time(NULL);

    while (!stopped)
    {
        /* j 키가 입력되면 측정을 중단합니다. */
        if (_kbhit())
        {
            key = read_key_press();

            if (key == 'j' || key == 'J')
                stopped = 1;
        }

        current_time = time(NULL);
        elapsed_seconds = difftime(current_time, start_time);
```

기능 추가(17) 스톱워치

```
    /* 기존 디지털 시계 함수에 맞춰 HHMMSS 형태로 변환합니다. */
    display_time =
        ((long)((long)elapsed_seconds / 3600) % 100) * 10000L
        + ((long)elapsed_seconds % 3600) / 60 * 100L
        + (long)elapsed_seconds % 60;

    system("cls");
    printf("===== 디지털 스톱워치 =====\n\n");
    print_digital_time(display_time);
    printf("\nj 키를 누르면 시간이 멈춥니다.\n");

    if (!stopped)
        wait_seconds(0.1);
}
```

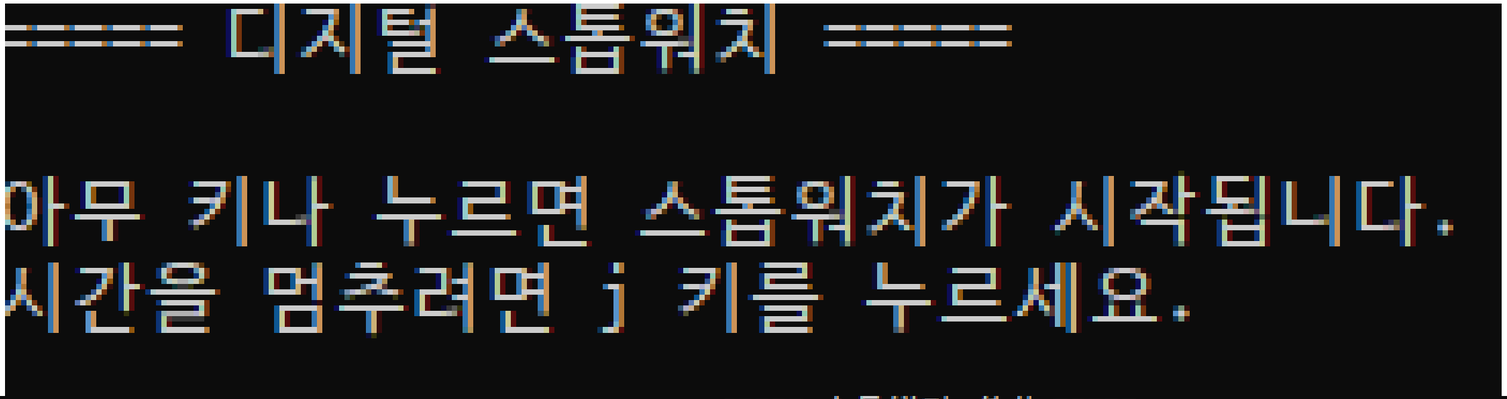
```
/* 멈춘 시각을 다시 계산하여 화면에 고정합니다. */
current_time = time(NULL);
elapsed_seconds = difftime(current_time, start_time);
display_time =
    ((long)((long)elapsed_seconds / 3600) % 100) * 10000L
    + ((long)elapsed_seconds % 3600) / 60 * 100L
    + (long)elapsed_seconds % 60;

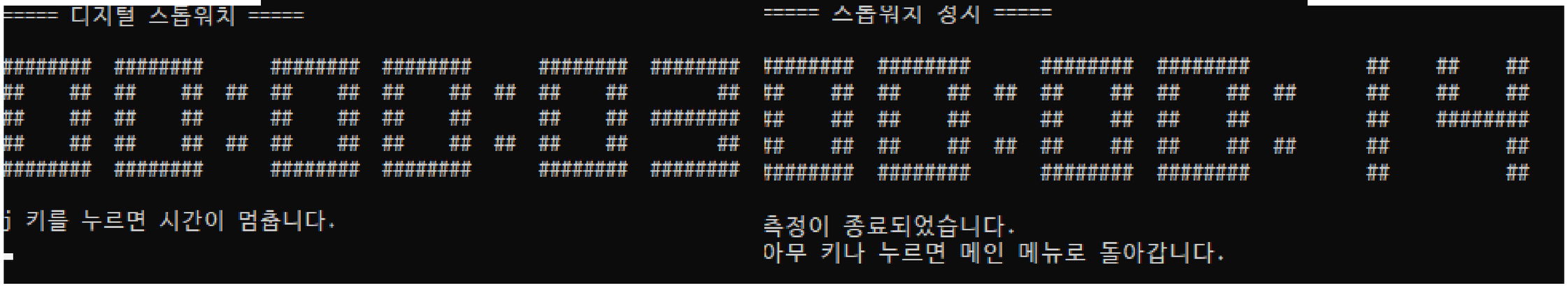
system("cls");
printf("===== 스톱워치 정지 =====\n\n");
print_digital_time(display_time);
printf("\n측정이 종료되었습니다.\n");
printf("아무 키나 누르면 메인 메뉴로 돌아갑니다.\n");

/* 정지 후 새로 입력되는 키를 기다립니다. */
while (_kbhit())
    read_key_press();

_getch();
```

기능 추가(17) 스톱워치


===== 디지털 스톱워치 =====
아무 키나 누르면 스톱워치가 시작됩니다.
시간을 멈추려면 j 키를 누르세요.


===== 디지털 스톱워치 =====
j 키를 누르면 시간이 멈춥니다.
===== 스톱워지 성시 =====
측정이 종료되었습니다.
아무 키나 누르면 메인 메뉴로 돌아갑니다.

기능 추가(18) 스톱워치-밀리초 표기

```c
/* 분:초:밀리초 형식으로 스톱워치 시간을 디지털 숫자로 출력합니다. */
void print_digital_stopwatch(long long elapsed_ms)
{
    int minutes;
    int seconds;
    int milliseconds;
    int digits[7];
    int row;
    int i;

    /* 경과 시간을 분, 초, 밀리초로 분리합니다. */
    minutes = (int)((elapsed_ms / 60000) % 100);
    seconds = (int)((elapsed_ms / 1000) % 60);
    milliseconds = (int)(elapsed_ms % 1000);

    digits[0] = minutes / 10;
    digits[1] = minutes % 10;
    digits[2] = seconds / 10;
    digits[3] = seconds % 10;
    digits[4] = milliseconds / 100;
    digits[5] = (milliseconds / 10) % 10;
    digits[6] = milliseconds % 10;
```

```c
/* 기존 디지털 숫자 모양을 재사용하여 MM:SS:MMM을 출력합니다. */
for (row = 0; row < 5; row++)
{
    for (i = 0; i < 7; i++)
    {
        print_digit_row(digits[i], row);

        if (i == 1 || i == 3)
        {
            printf("  ");

            /* 콜론을 디지털 점 두 개로 표시합니다. */
            if (row == 1 || row == 3)
                printf("##");
            else
                printf("  ");

            printf("  ");
        }
        else if (i != 6)
        {
            printf("  ");
        }
    }

    printf("\n");
}
```

기능 추가(18) 스톱워치-밀리초 표시

```
    start_time = StopwatchClock::now();
```

```
elapsed_ms = std::chrono::duration_cast<std::chrono::milliseconds>(
    StopwatchClock::now() - start_time).count();
```

```
/* 10밀리초 간격으로 화면을 갱신합니다. */
std::this_thread::sleep_for(std::chrono::milliseconds(10));
```

```
===== 스톱워치 정지 =====

########  ########     ########  ########        ##    ########  ########
##    ##  ##    ##  ##  ##    ##  ##      ##      ##    ##    ##  ##    ##
##    ##  ##    ##      ##    ##  ########        ##    ########  ########
##    ##  ##    ##  ##  ##    ##        ##  ##    ##    ##              ##
########  ########     ########  ########        ##    ########        ##

측정이 종료되었습니다.
아무 키나 누르면 메인 메뉴로 돌아갑니다.
```

기능 추가(19) 스톱워치-재시작, 초기화

```
long long elapsed_ms = 0;           //화면에 표시할 현재 경과 시간
long long accumulated_ms = 0;       //일시 정지하기 전까지 누적된 시간
int key;
int running = 0;                    //스톱워치가 작동 중인지 정지 중인지 구분
```

```
current_time = StopwatchClock::now();
elapsed_ms = accumulated_ms +
    std::chrono::duration_cast<std::chrono::milliseconds>(
        current_time - start_time).count();
/* 시작을 알리는 첫 번째 키 입력은 측정에 포함하지 않습니다. */
read_key_press();
start_time = StopwatchClock::now();
running = 1;
```

```
/* 누적 시간을 0으로 초기화하고 정지 상태를 유지합니다. */
accumulated_ms = 0;
elapsed_ms = 0;
```

기능 추가(19) 스톱워치-재시작, 초기화

```
===== 디지털 스톱워치 =====

아무 키나 누르면 스톱워치가 시작됩니다.
작동 중 j 키: 일시 정지
정지 중 k 키: 다시 시작
정지 중 j 키: 0으로 초기화
정지 중 0 키: 메인 메뉴로 돌아가기
```

```
===== 스톱워치 일시 정지 =====

########  ########     ########  ########     ########  ########  ########
##    ##  ##    ##  ##  ##    ##        ##  ##  ##    ##        ##  ##    ##
##    ##  ##    ##      ##    ##  ########      ########  ########  ########
##    ##  ##    ##  ##  ##    ##        ##  ##  ##    ##  ##              ##
########  ########     ########  ########     ########  ########        ##

정지 상태에서 선택하세요.
j : 초기화
k : 다시 시작
0 : 메인 메뉴로 돌아가기
```

```
===== 스톱워치 초기화 =====

########  ########     ########  ########     ########  ########  ########
##    ##  ##    ##  ##  ##    ##        ##  ##  ##    ##        ##  ##    ##
##    ##  ##    ##      ##    ##  ########      ########  ########  ########
##    ##  ##    ##  ##  ##    ##        ##  ##  ##    ##  ##              ##
########  ########     ########  ########     ########  ########        ##

초기화되었습니다. 현재 정지 상태입니다.
k : 시작
0 : 메인 메뉴로 돌아가기
j : 다시 초기화
```

기능 추가(20) 스톱워치-구간 기록 저장

```c
/* 저장된 구간 기록을 출력합니다. */
void print_lap_records(long long lap_times[],
                       long long total_times[],
                       int lap_count)
{
    int i;
    long long lap_minutes;
    long long lap_seconds;
    long long lap_milliseconds;
    long long total_minutes;
    long long total_seconds;
    long long total_milliseconds;

    printf("\n===== 구간 기록 =====\n\n");

    if (lap_count == 0)
    {
        printf("저장된 구간 기록이 없습니다.\n");
        return;
    }

    for (i = 0; i < lap_count; i++)
    {
        lap_minutes = (lap_times[i] / 60000) % 100;
        lap_seconds = (lap_times[i] / 1000) % 60;
        lap_milliseconds = lap_times[i] % 1000;

        total_minutes = (total_times[i] / 60000) % 100;
        total_seconds = (total_times[i] / 1000) % 60;
        total_milliseconds = total_times[i] % 1000;

        printf("Lap %d | ", i + 1);

        printf("구간 시간: %02lld:%02lld:%03lld | ",
               lap_minutes, lap_seconds, lap_milliseconds);

        printf("누적 시간: %02lld:%02lld:%03lld\n",
               total_minutes, total_seconds, total_milliseconds);
    }
```

기능 추가(20) 스톱워치-구간 기록 저장

```
/* 구간 기록을 저장할 배열입니다. */
long long lap_times[100];
long long total_times[100];

/* 직전 구간 기록 시점의 누적 시간입니다. */
long long last_lap_ms = 0;

int lap_count = 0;
```

```
else if (key == 'l' || key == 'L')
{
    /* 현재까지의 경과 시간을 계산합니다. */
    current_time = StopwatchClock::now();

    elapsed_ms = accumulated_ms +
        std::chrono::duration_cast<std::chrono::milliseconds>(
            current_time - start_time).count();

    /* 구간 기록을 배열에 저장합니다. */
    if (lap_count < 100)
    {
        lap_times[lap_count] = elapsed_ms - last_lap_ms;
        total_times[lap_count] = elapsed_ms;

        last_lap_ms = elapsed_ms;
        lap_count++;
    }
}
```

```
else if (key == 'l' || key == 'L')
{
    /* 정지 상태에서는 현재 누적 시간을 기록합니다. */
    elapsed_ms = accumulated_ms;

    if (lap_count < 100)
    {
        lap_times[lap_count] = elapsed_ms - last_lap_ms;
        total_times[lap_count] = elapsed_ms;

        last_lap_ms = elapsed_ms;
        lap_count++;
    }

    system("cls");

    printf("===== 스톱워치 일시 정지 =====\n\n");

    print_digital_stopwatch(accumulated_ms);

    print_lap_records(lap_times, total_times, lap_count);

    printf("\nk : 다시 시작\n");
    printf("j : 초기화\n");
    printf("l : 구간 기록\n");
    printf("0 : 메인 메뉴로 돌아가기\n");
```

기능 추가(20) 스톱워치-구간 기록 저장

```
===== 스톱워치 일시 정지 =====

########  ########     ########  ########     ########  ########  ########
##    ##  ##    ##  ##  ##    ##  ##    ##  ##        ##    ##  ##    ##
##    ##  ##    ##      ##    ##  ########     ########  ########  ##    ##
##    ##  ##    ##  ##  ##    ##        ##  ##        ##  ##    ##  ##    ##
########  ########     ########        ##     ########  ########  ########

===== 구간 기록 =====

Lap 1 ¦ 구간 시간: 00:02:701 ¦ 누적 시간: 00:02:701
Lap 2 ¦ 구간 시간: 00:02:010 ¦ 누적 시간: 00:04:711
Lap 3 ¦ 구간 시간: 00:01:957 ¦ 누적 시간: 00:06:668
Lap 4 ¦ 구간 시간: 00:00:000 ¦ 누적 시간: 00:06:668
Lap 5 ¦ 구간 시간: 00:00:893 ¦ 누적 시간: 00:07:561
Lap 6 ¦ 구간 시간: 00:01:819 ¦ 누적 시간: 00:09:380

k : 다시 시작
j : 초기화
l : 구간 기록
0 : 메인 메뉴로 돌아가기
```

기능 추가(21) 디지털 블랙잭

```c
/* 블랙잭 카드 한 장을 출력합니다. */
void print_blackjack_card(int rank, int suit)
{
    const char *rank_names[14] =
    {
        "", "A", "2", "3", "4", "5", "6",
        "7", "8", "9", "10", "J", "Q", "K"
    };

    const char *suit_names[4] =
    {
        "S", "H", "D", "C"
    };

    printf("+-------+\n");
    printf("| %-2s    |\n", rank_names[rank]);
    printf("|   %s   |\n", suit_names[suit]);
    printf("|    %-2s |\n", rank_names[rank]);
    printf("+-------+\n");
}
```

```c
/* 블랙잭 카드 패를 출력합니다. */
void print_blackjack_hand(int cards[], int suits[], int count)
{
    int i;

    for (i = 0; i < count; i++)
    {
        print_blackjack_card(cards[i], suits[i]);
        printf("\n");
    }
}
```

기능 추가(21) 디지털 블랙잭

```c
/* 블랙잭 점수를 기존 디지털 숫자 출력 함수로 표시합니다. */
void print_blackjack_score(int score)
{
    int digits[2];
    int row;
    int i;

    digits[0] = score / 10;
    digits[1] = score % 10;

    for (row = 0; row < 5; row++)
    {
        for (i = 0; i < 2; i++)
        {
            print_digit_row(digits[i], row);

            if (i == 0)
                printf("  ");
        }

        printf("\n");
    }
}
```

```c
/* 블랙잭 카드 패의 합계를 계산합니다. */
int calculate_blackjack_score(int cards[], int count)
{
    int total = 0;
    int ace_count = 0;
    int i;
    int value;

    for (i = 0; i < count; i++)
    {
        if (cards[i] == 1)
        {
            /* 에이스는 우선 11점으로 계산합니다. */
            total += 11;
            ace_count++;
        }
        else if (cards[i] >= 11)
        {
            /* J, Q, K는 10점입니다. */
            total += 10;
        }
        else
        {
            total += cards[i];
        }
    }

    /* 21점을 초과하면 에이스를 11점에서 1점으로 변경합니다. */
    while (total > 21 && ace_count > 0)
    {
        total -= 10;
        ace_count--;
    }

    return total;
}
```

```c
/* 6번 모드: 디지털 블랙잭 게임입니다. */
void play_blackjack_mode(void)
{
    int deck[52];
    int deck_suits[52];

    int player_cards[12];
    int player_suits[12];
    int dealer_cards[12];
    int dealer_suits[12];

    int deck_index = 0;
    int player_count = 0;
    int dealer_count = 0;

    int i;
    int j;
    int temp;

    int player_score;
    int dealer_score;
    int choice;
    int ch;

    int player_natural;
    int dealer_natural;
    int player_bust = 0;
```

기능 추가(21) 디지털 블랙잭

```
/* 52장의 카드 덱을 생성합니다. */
for (i = 0; i < 52; i++)
{
    deck[i] = (i % 13) + 1;
    deck_suits[i] = i / 13;
}

/* Fisher-Yates 방식으로 카드 덱을 섞습니다. */
for (i = 51; i > 0; i--)
{
    j = rand() % (i + 1);

    temp = deck[i];
    deck[i] = deck[j];
    deck[j] = temp;

    temp = deck_suits[i];
    deck_suits[i] = deck_suits[j];
    deck_suits[j] = temp;
}
```

```
/* 플레이어와 딜러에게 번갈아 두 장씩 배분합니다. */
player_cards[player_count] = deck[deck_index];
player_suits[player_count] = deck_suits[deck_index];
player_count++;
deck_index++;

dealer_cards[dealer_count] = deck[deck_index];
dealer_suits[dealer_count] = deck_suits[deck_index];
dealer_count++;
deck_index++;

player_cards[player_count] = deck[deck_index];
player_suits[player_count] = deck_suits[deck_index];
player_count++;
deck_index++;

dealer_cards[dealer_count] = deck[deck_index];
dealer_suits[dealer_count] = deck_suits[deck_index];
dealer_count++;
deck_index++;

player_score = calculate_blackjack_score(player_cards, player_count);
dealer_score = calculate_blackjack_score(dealer_cards, dealer_count);

player_natural = (player_score == 21);
dealer_natural = (dealer_score == 21);
```

기능 추가(21) 디지털 블랙잭

```c
/* 플레이어의 선택 단계입니다. */
while (1)
{
    system("cls");

    printf("====================================\n");
    printf("          DIGITAL BLACKJACK\n");
    printf("====================================\n\n");

    printf("[ DEALER ]\n\n");

    print_blackjack_card(dealer_cards[0], dealer_suits[0]);

    if (dealer_count >= 2)
    {
        printf("+-------+\n");
        printf("|   ?   |\n");
        printf("|  ???  |\n");
        printf("|   ?   |\n");
        printf("+-------+\n");
    }

    printf("\n공개된 카드 점수: ");

    if (dealer_cards[0] == 1)
        printf("1 또는 11\n");
    else if (dealer_cards[0] >= 10)
        printf("10\n");
    else
        printf("%d\n", dealer_cards[0]);

    printf("\n====================================\n\n");

    printf("[ PLAYER ]\n\n");

    print_blackjack_hand(player_cards, player_suits, player_count);

    player_score = calculate_blackjack_score(player_cards, player_count);

    printf("PLAYER SCORE\n\n");
    print_blackjack_score(player_score);

    printf("\n점수: %d\n", player_score);

    printf("\n====================================\n");

    /* 플레이어가 21점을 초과하면 즉시 패배합니다. */
    if (player_score > 21)
    {
        player_bust = 1;
        break;
    }

    /* 최초 두 장으로 21점을 만들었다면 블랙잭입니다. */
    if (player_natural)
        break;

    printf("\n1. HIT\n");
    printf("2. STAND\n");
    printf("0. 메뉴로 돌아가기\n\n");
    printf("선택: ");

    if (scanf("%d", &choice) != 1)
    {
        printf("잘못된 입력입니다. 숫자를 입력하세요.\n");

        while ((ch = getchar()) != '\n' && ch != EOF)
        {
        }

        wait_seconds(1.0);
        continue;
    }

    while ((ch = getchar()) != '\n' && ch != EOF)
    {
    }

    if (choice == 0)
        return;

    if (choice == 1)
    {
        /* 카드를 한 장 더 받습니다. */
        if (player_count < 12 && deck_index < 52)
        {
            player_cards[player_count] = deck[deck_index];
            player_suits[player_count] = deck_suits[deck_index];

            player_count++;
            deck_index++;
        }
        else
        {
            printf("더 이상 카드를 받을 수 없습니다.\n");
            wait_seconds(1.0);
            break;
        }
    }
    else if (choice == 2)
    {
        /* 현재 카드로 승부를 진행합니다. */
        break;
    }
    else
    {
        printf("잘못된 선택입니다.\n");
        wait_seconds(1.0);
    }
}
```

기능 추가(21) 디지털 블랙잭

```c
/* 플레이어가 버스트하지 않았고 블랙잭도 아닌 경우 딜러가 진행합니다. */
if (!player_bust && !player_natural)
{
    while (1)
    {
        dealer_score =
            calculate_blackjack_score(dealer_cards, dealer_count);

        /* 딜러는 17점 이상이면 카드를 받지 않습니다. */
        if (dealer_score >= 17)
            break;

        if (dealer_count >= 12 || deck_index >= 52)
            break;

        dealer_cards[dealer_count] = deck[deck_index];
        dealer_suits[dealer_count] = deck_suits[deck_index];

        dealer_count++;
        deck_index++;
    }
}
```

```c
/* 최종 결과를 출력합니다. */
system("cls");

player_score = calculate_blackjack_score(player_cards, player_count);
dealer_score = calculate_blackjack_score(dealer_cards, dealer_count);

printf("====================================\n");
printf("          BLACKJACK RESULT\n");
printf("====================================\n\n");

printf("[ DEALER ]\n\n");
print_blackjack_hand(dealer_cards, dealer_suits, dealer_count);

printf("DEALER SCORE\n\n");
print_blackjack_score(dealer_score);
printf("\n점수: %d\n", dealer_score);

printf("\n====================================\n\n");

printf("[ PLAYER ]\n\n");
print_blackjack_hand(player_cards, player_suits, player_count);

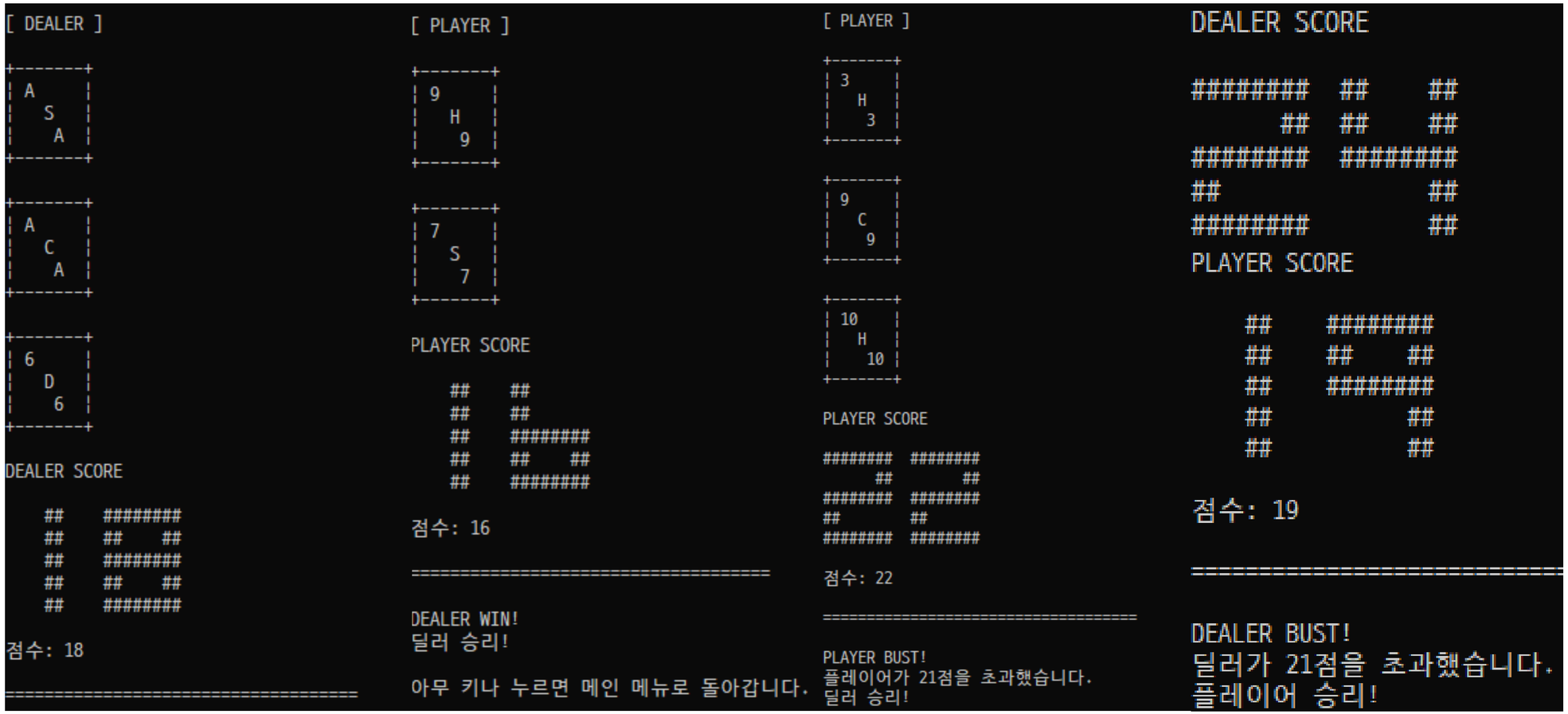
printf("PLAYER SCORE\n\n");
print_blackjack_score(player_score);
printf("\n점수: %d\n", player_score);

printf("\n====================================\n\n");
```

```c
/* 블랙잭 및 승패를 판정합니다. */
if (player_bust)
{
    printf("PLAYER BUST!\n");
    printf("플레이어가 21점을 초과했습니다.\n");
    printf("딜러 승리!\n");
}
else if (player_natural && dealer_natural)
{
    printf("BOTH BLACKJACK!\n");
    printf("무승부입니다.\n");
}
else if (player_natural)
{
    printf("BLACKJACK!\n");
    printf("플레이어 승리!\n");
}
else if (dealer_natural)
{
    printf("DEALER BLACKJACK!\n");
    printf("딜러 승리!\n");
}
else if (dealer_score > 21)
{
    printf("DEALER BUST!\n");
    printf("딜러가 21점을 초과했습니다.\n");
    printf("플레이어 승리!\n");
}
else if (player_score > dealer_score)
{
    printf("PLAYER WIN!\n");
    printf("플레이어 승리!\n");
}
else if (player_score < dealer_score)
{
    printf("DEALER WIN!\n");
    printf("딜러 승리!\n");
}
else
{
    printf("DRAW!\n");
    printf("무승부입니다.\n");
}

printf("\n아무 키나 누르면 메인 메뉴로 돌아갑니다.\n");
_getch();
```

기능 추가(21) 디지털 블랙잭



기능 추가(22) 20초 맞히기

```c
/* 분:초:밀리초를 디지털 숫자로 출력합니다.
   hide_seconds가 1이면 초의 두 자리를 숨깁니다. */
void print_digital_accuracy_time(long long elapsed_ms,
                                 int hide_seconds)
{
    int minutes;
    int seconds;
    int milliseconds;
    int digits[7];
    int row;
    int i;

    minutes = (int)((elapsed_ms / 60000) % 100);
    seconds = (int)((elapsed_ms / 1000) % 60);
    milliseconds = (int)(elapsed_ms % 1000);

    digits[0] = minutes / 10;
    digits[1] = minutes % 10;
    digits[2] = seconds / 10;
    digits[3] = seconds % 10;
    digits[4] = milliseconds / 100;
    digits[5] = (milliseconds / 10) % 10;
    digits[6] = milliseconds % 10;
```

```c
for (row = 0; row < 5; row++)
{
    for (i = 0; i < 7; i++)
    {
        /* 3초가 지난 뒤에는 초의 두 자리를 공백으로 출력합니다. */
        if (hide_seconds && (i == 2 || i == 3))
            printf("        ");
        else
            print_digit_row(digits[i], row);

        if (i == 1 || i == 3)
        {
            printf("  ");

            if (row == 1 || row == 3)
                printf("##");
            else
                printf("  ");

            printf("  ");
        }
        else if (i != 6)
        {
            printf("  ");
        }
    }

    printf("\n");
}
```

```c
/* 20초에 최대 500밀리초의 오차를 허용하는 게임입니다. */
void play_20_second_game(void)
{
    typedef std::chrono::steady_clock GameClock;

    GameClock::time_point start_time;
    GameClock::time_point current_time;

    long long elapsed_ms;
    long long difference_ms;

    int key;
    int hide_seconds;

    system("cls");

    printf("===== 20초 맞히기 게임 =====\n\n");
    printf("목표 시간: 20초\n");
    printf("허용 오차: ±500밀리초\n");
    printf("성공 범위: 00:19:501 ~ 00:20:500\n\n");
    printf("아무 키나 누르면 측정이 시작됩니다.\n");
    printf("측정 중 아무 키나 누르면 시간이 멈춥니다.\n");
    printf("3초가 지나면 디지털 표기에서 초가 사라집니다.\n");

    /* 시작 키 입력은 측정 시간에 포함하지 않습니다. */
    read_key_press();
    start_time = GameClock::now();
```

기능 추가(22) 20초 맞히기

```
/* 시작 키 입력은 측정 시간에 포함하지 않습니다. */
read_key_press();
start_time = GameClock::now();

while (1)
{
    current_time = GameClock::now();

    elapsed_ms =
        std::chrono::duration_cast<std::chrono::milliseconds>(
            current_time - start_time).count();

    /* 3초 이상이면 디지털 시간 전체를 숨깁니다. */
    hide_seconds = (elapsed_ms >= 3000);

    system("cls");

    printf("===== 20초 맞히기 게임 =====\n\n");

    if (hide_seconds == 0)
    {
        printf("목표 시간: 00:20:000\n\n");
        print_digital_accuracy_time(elapsed_ms, 0);
    }
    else
    {
        printf("목표 시간: 비공개\n\n");
        printf("시간을 확인하지 말고 20초를 맞혀보세요!\n");
    }

    printf("\n아무 키나 누르면 시간을 멈춥니다.\n");

    /* 키 입력이 들어오면 측정을 종료합니다. */
    if (_kbhit())
    {
        key = read_key_press();
        (void)key;

        current_time = GameClock::now();

        elapsed_ms =
            std::chrono::duration_cast<std::chrono::milliseconds>(
                current_time - start_time).count();

        break;
    }

    std::this_thread::sleep_for(std::chrono::milliseconds(10));
}
```

```
/* 최종 결과에서는 초를 다시 표시합니다. */
system("cls");

printf("===== 20초 맞히기 게임 결과 =====\n\n");

print_digital_accuracy_time(elapsed_ms, 0);

difference_ms = elapsed_ms - 20000;

printf("\n측정 시간: %02lld:%02lld:%03lld\n",
       (elapsed_ms / 60000) % 100,
       (elapsed_ms / 1000) % 60,
       elapsed_ms % 1000);

if (elapsed_ms >= 19501 && elapsed_ms <= 20500)
{
    printf("\n성공! 목표 범위 시간을 맞혔습니다.\n");
}
else if (elapsed_ms < 19501)
{
    printf("\n실패! 너무 일찍 멈췄습니다.\n");
}
else
{
    printf("\n실패! 너무 늦게 멈췄습니다.\n");
}

if (difference_ms > 0)
    printf("오차 : %lld밀리초\n", difference_ms);
else if (difference_ms < 0)
    printf("오차 : -%lld밀리초\n", -difference_ms);
else
    printf("목표 시간과 정확히 일치합니다!\n");

printf("\n아무 키나 누르면 메인 메뉴로 돌아갑니다.\n");
read_key_press();
```

기능 추가(22) 20초 맞히기

```
===== 20초 맞히기 게임 =====

목표 시간: 20초
허용 오차: ±500밀리초
성공 범위: 00:19:501 ~ 00:20:500

아무 키나 누르면 측정이 시작됩니다.
측정 중 아무 키나 누르면 시간이 멈춥니다.
3초가 지나면 디지털 표기에서 초가 사라집니다.
```

```
===== 20초 맞히기 게임 결과 =====

########  ########     ########  ########     ########  ########  ########
##    ##  ##    ## ##         ##  ##    ##  ##  ##    ##  ##    ##  ##    ##
##    ##  ##    ##     ########  ##    ##      ##    ##  ########  ########
##    ##  ##    ## ##  ##        ##    ##  ##  ##    ##        ##  ##    ##
########  ########     ########  ########     ########        ##  ########

측정 시간: 00:20:098

성공! 목표 범위 시간을 맞혔습니다.
오차 : 98밀리초

아무 키나 누르면 메인 메뉴로 돌아갑니다.
===== 20초 맞히기 게임 결과 =====

########  ########     ########     ##          ##    ##    ##  ########
##    ##  ##    ## ##         ##     ##    ##    ##    ##    ##  ##    ##
##    ##  ##    ##     ########     ##          ##    ########  ########
##    ##  ##    ## ##  ##           ##    ##    ##          ##  ##    ##
########  ########     ########     ##          ##          ##  ########

측정 시간: 00:21:148

실패! 너무 늦게 멈췄습니다.
목표 시간보다 1148밀리초 늦었습니다.

아무 키나 누르면 메인 메뉴로 돌아갑니다.
```

기능 추가(23) 20초 맞히기-난이도

```c
/* 1부터 4까지의 난이도를 입력받습니다. */
while (1)
{
    if (scanf("%d", &difficulty) != 1)
    {
        printf("잘못된 입력입니다. 1부터 4까지 입력하세요: ");

        while ((ch = getchar()) != '\n' && ch != EOF)
        {
        }

        continue;
    }

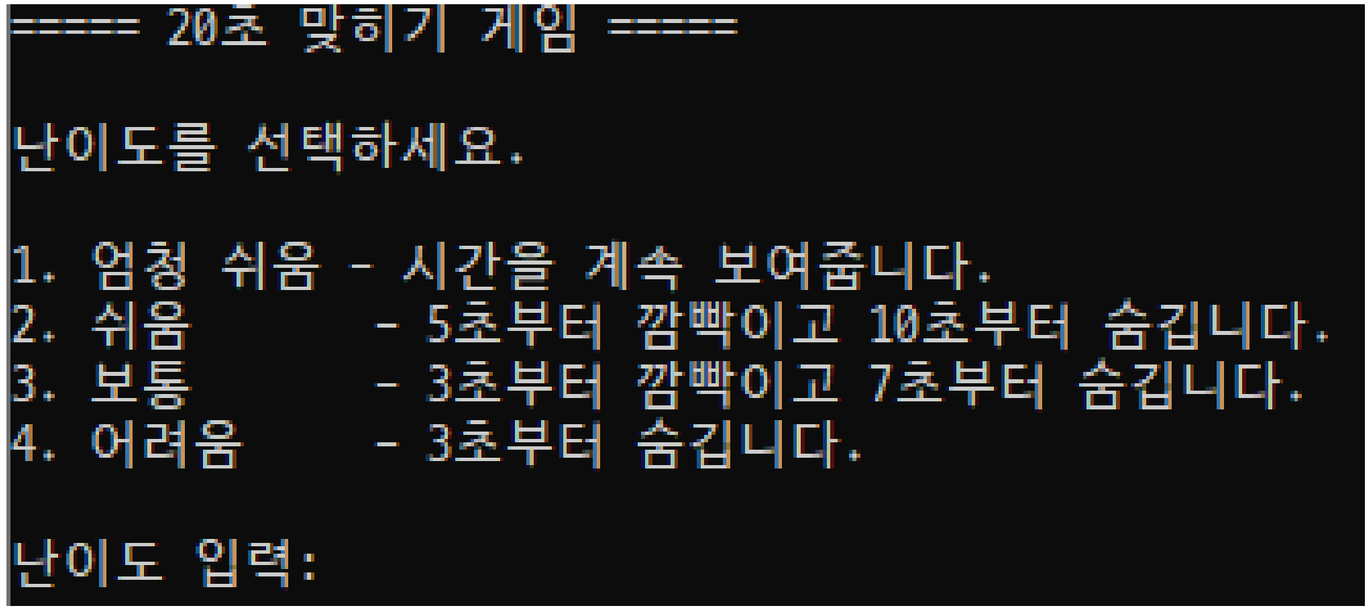
    while ((ch = getchar()) != '\n' && ch != EOF)
    {
    }

    if (difficulty >= 1 && difficulty <= 4)
        break;

    printf("잘못된 난이도입니다. 1부터 4까지 입력하세요: ");
}
```

```c
/* 난이도에 따라 깜빡임 시작 시간과 숨김 시간을 설정합니다. */
if (difficulty == 1)
{
    blink_start_ms = -1;
    hide_after_ms = -1;
}
else if (difficulty == 2)
{
    blink_start_ms = 5000;
    hide_after_ms = 10000;
}
else if (difficulty == 3)
{
    blink_start_ms = 3000;
    hide_after_ms = 7000;
}
else
{
    blink_start_ms = -1;
    hide_after_ms = 3000;
}
```

기능 추가(23) 20초 맞히기-난이도



기능 추가(24) 디지털 비밀금고

```c
/* 네 자리 숫자를 디지털 형태로 출력합니다. */
void print_digital_vault_code(const char code[])
{
    int row;
    int i;

    for (row = 0; row < 5; row++)
    {
        for (i = 0; i < 4; i++)
        {
            print_digit_row(code[i] - '0', row);

            if (i < 3)
                printf("  ");
        }

        printf("\n");
    }
}
```

```c
/* 8번 모드: 네 자리 디지털 비밀금고 게임입니다. */
void play_digital_vault_game(void)
{
    int digits[10];
    int i;
    int j;
    int temp;
    int ch;

    int attempts = 0;
    int is_correct;
    int valid;

    char secret[5];
    char display[5] = "0000";
    char guess[6];
```

기능 추가(24) 디지털 비밀금고

```c
/* 0부터 9까지의 숫자를 배열에 저장합니다. */
for (i = 0; i < 10; i++)
{
    digits[i] = i;
}

/* Fisher-Yates 방식으로 숫자를 무작위로 섞습니다. */
for (i = 9; i > 0; i--)
{
    j = rand() % (i + 1);

    temp = digits[i];
    digits[i] = digits[j];
    digits[j] = temp;
}

/* 섞인 숫자 중 앞의 네 개를 비밀번호로 사용합니다. */
for (i = 0; i < 4; i++)
{
    secret[i] = (char)('0' + digits[i]);
}

secret[4] = '\0';
```

```c
while (1)
{
    system("cls");

    printf("===== DIGITAL SECRET VAULT =====\n\n");
    printf("네 자리 비밀번호를 맞혀 금고를 여세요!\n");
    printf("비밀번호는 0부터 9까지의 숫자로 구성됩니다.\n");
    printf("각 숫자는 중복되지 않습니다.\n\n");

    printf("현재 금고 입력 번호\n\n");
    print_digital_vault_code(display);

    printf("\n시도 횟수: %d회\n", attempts);
    printf("\n네 자리 번호 입력: ");

    /* 네 자리 번호를 입력받습니다. */
    if (scanf("%5s", guess) != 1)
        return;

    /* 입력 뒤에 남은 문자를 비웁니다. */
    while ((ch = getchar()) != '\n' && ch != EOF)
    {
    }
```

기능 추가(24) 디지털 비밀금고

```c
/* 입력한 값이 정확히 네 자리 숫자인지 검사합니다. */
valid = 1;

if (guess[4] != '\0')
{
    valid = 0;
}
else
{
    for (i = 0; i < 4; i++)
    {
        if (guess[i] < '0' || guess[i] > '9')
        {
            valid = 0;
            break;
        }
    }
}

if (!valid)
{
    printf("\n잘못된 입력입니다. 네 자리 숫자를 입력하세요.\n");
    wait_seconds(1.0);
    continue;
}
```

```c
/* 입력한 번호를 현재 디지털 표기에 반영합니다. */
for (i = 0; i < 4; i++)
{
    display[i] = guess[i];
}

display[4] = '\0';

attempts++;

/* 입력한 번호와 비밀번호를 비교합니다. */
is_correct = 1;

for (i = 0; i < 4; i++)
{
    if (guess[i] != secret[i])
    {
        is_correct = 0;
        break;
    }
}

system("cls");
```

```c
    printf("===== DIGITAL SECRET VAULT =====\n\n");
    printf("현재 금고 입력 번호\n\n");

    print_digital_vault_code(display);

    printf("\n시도 횟수: %d회\n", attempts);

    if (is_correct)
    {
        printf("\n==================================\n");
        printf("          VAULT OPEN!\n");
        printf("==================================\n\n");

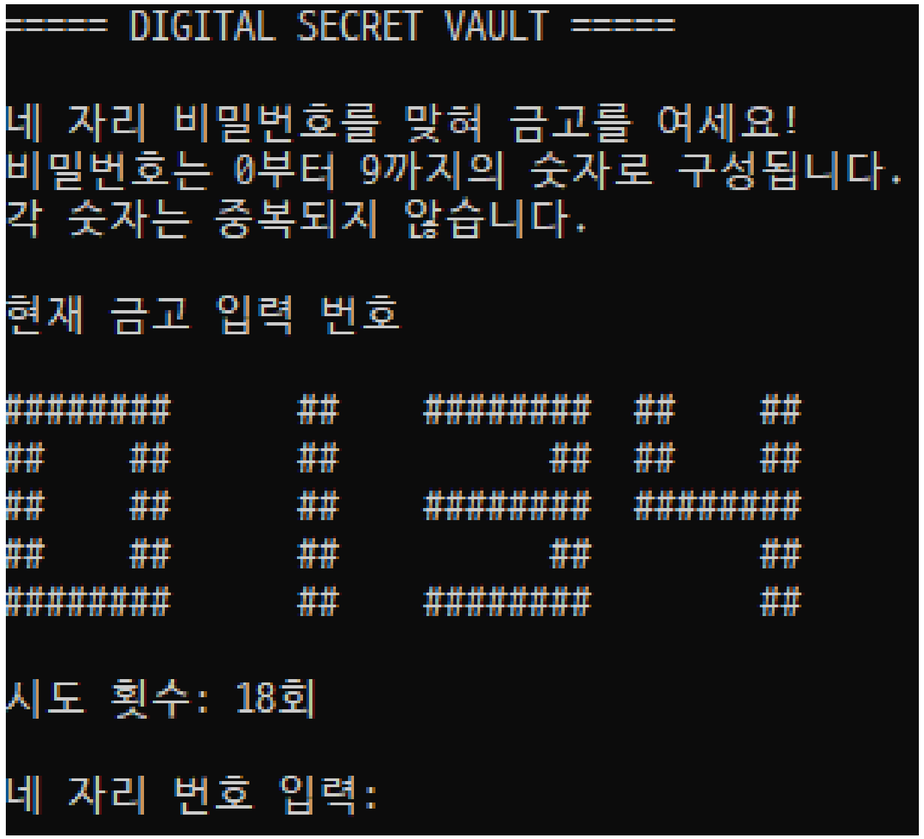
        printf("축하합니다! 비밀금고를 열었습니다!\n");
        printf("비밀번호: %s\n", secret);
        printf("총 시도 횟수: %d회\n", attempts);

        break;
    }
    else
    {
        printf("\n비밀번호가 틀렸습니다.\n");
        printf("다시 시도해 주세요.\n");

        wait_seconds(1.0);
    }
}

printf("\n아무 키나 누르면 메인 메뉴로 돌아갑니다.\n");
read_key_press();
```

기능 추가(24) 디지털 비밀금고



기능 추가(25) 디지털 비밀금고 - 힌트

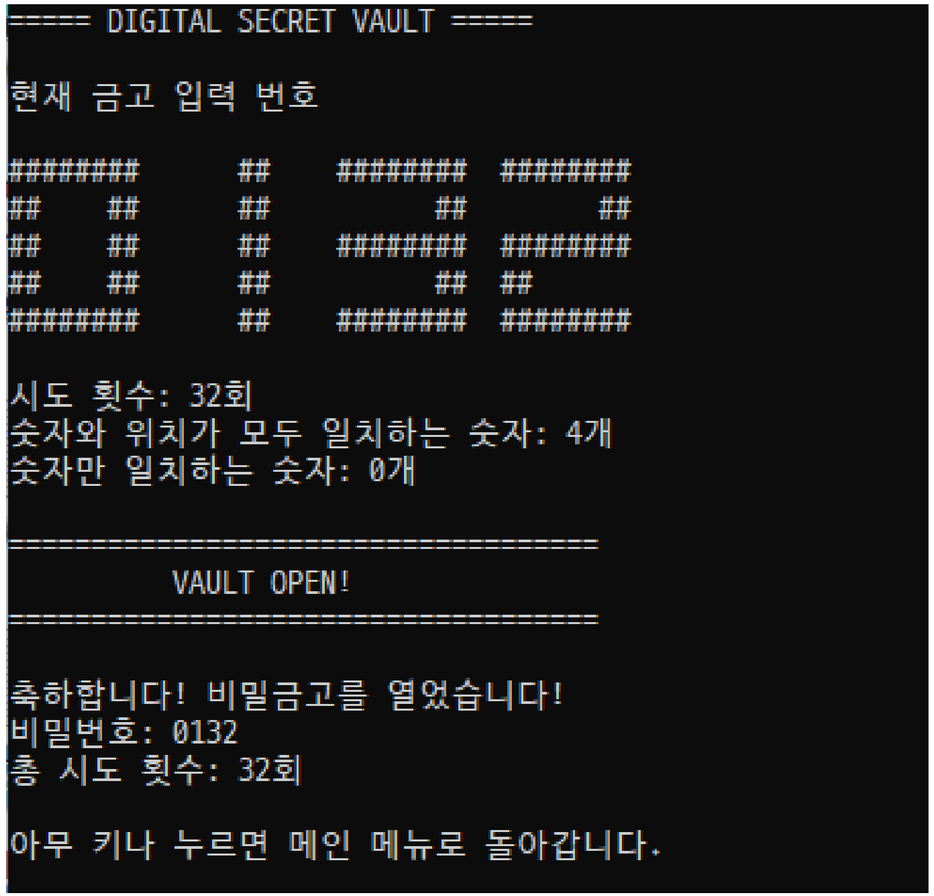
```c
/* 입력한 번호와 비밀번호를 비교합니다. */
is_correct = 1;
correct_position = 0;
correct_number = 0;

/* 숫자와 위치가 모두 일치하는 개수를 계산합니다. */
for (i = 0; i < 4; i++)
{
    matched[i] = 0;

    if (guess[i] == secret[i])
    {
        correct_position++;
        matched[i] = 1;
    }
    else
    {
        is_correct = 0;
    }
}
```

```c
/*
   숫자만 일치하고 위치가 다른 개수를 계산합니다.
   이미 일치한 비밀번호 숫자는 중복해서 세지 않습니다.
*/
for (i = 0; i < 4; i++)
{
    if (guess[i] != secret[i])
    {
        for (j = 0; j < 4; j++)
        {
            if (!matched[j] && guess[i] == secret[j])
            {
                correct_number++;
                matched[j] = 1;
                break;
            }
        }
    }
}
/* 직전 시도의 힌트를 다음 입력 화면에도 표시합니다. */
if (attempts > 0)
{
    printf("숫자와 위치가 모두 일치하는 숫자: %d개\n",
           correct_position);
    printf("숫자만 일치하는 숫자: %d개\n",
           correct_number);
}
```

기능 추가(25) 디지털 비밀금고 - 힌트



기능 추가(26) 디지털 비밀금고 - 시도 기록

```
/* 디지털 비밀금고 시도 기록 구조체 */
struct VaultAttempt
{
    char guess[5];
    int correct_position;
    int correct_number;
};

/* 이번 시도의 번호와 힌트를 기록에 추가합니다. */
for (i = 0; i < 4; i++)
{
    current_attempt.guess[i] = guess[i];
}
current_attempt.guess[4] = '\0';
current_attempt.correct_position = correct_position;
current_attempt.correct_number = correct_number;
history.push_back(current_attempt);
```

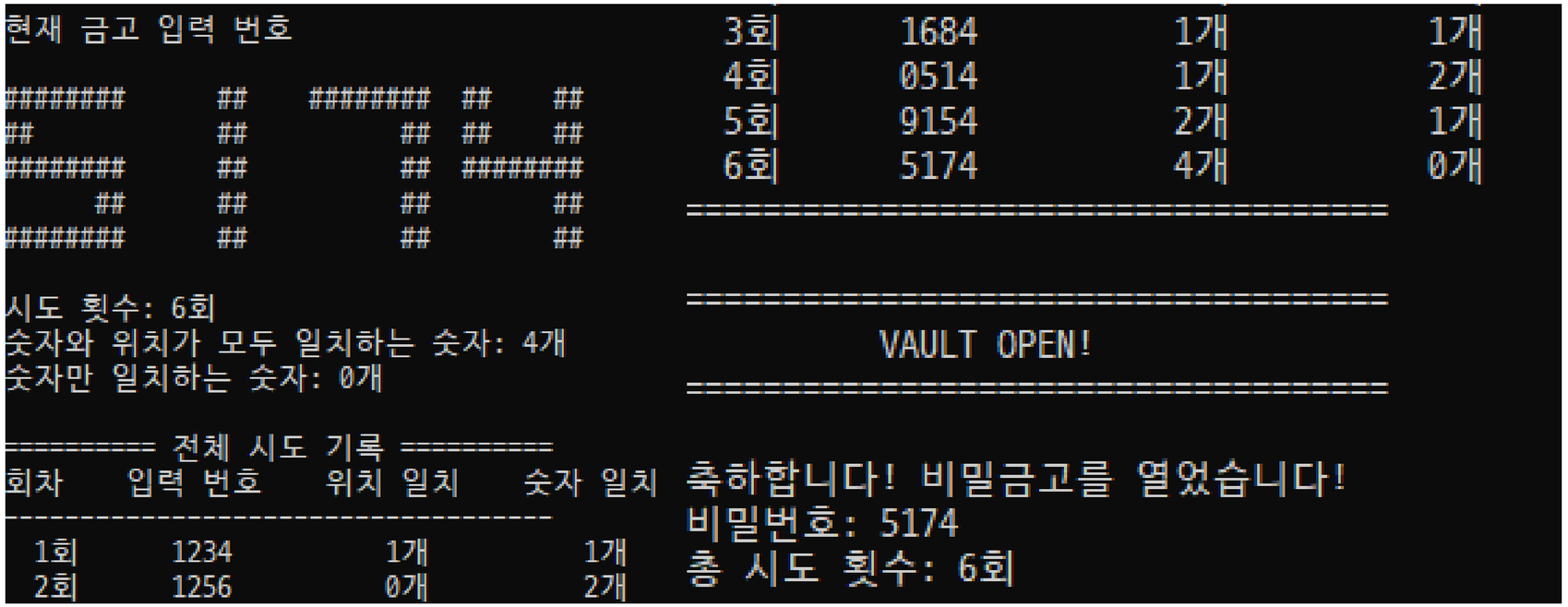
```
/* 유효한 입력을 모두 저장하는 기록입니다. */
std::vector<VaultAttempt> history;
VaultAttempt current_attempt;

/* 방금 입력한 시도를 포함하여 전체 기록을 표시합니다. */
printf("\n========== 전체 시도 기록 ==========\n");
printf("회차    입력 번호    위치 일치    숫자 일치\n");
printf("-----------------------------------\n");

for (i = 0; i < (int)history.size(); i++)
{
    printf("%3d회      %s          %d개          %d개\n",
           i + 1,
           history[i].guess,
           history[i].correct_position,
           history[i].correct_number);
}

printf("===================================\n");
```

기능 추가(26) 디지털 비밀금고 - 시도 기록



기능 추가(27) 디지털 비밀금고 - 색깔 힌트

```c
/* 네 자리 숫자를 디지털 형태로 출력합니다. */
void print_digital_vault_code(const char code[], const int colors[])
{
    HANDLE console = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_SCREEN_BUFFER_INFO console_info;
    WORD default_color = FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_BLUE;
    int row;
    int i;

    /* 현재 콘솔의 기본 글자색을 저장합니다. */
    if (GetConsoleScreenBufferInfo(console, &console_info))
    {
        default_color = console_info.wAttributes;
    }

    for (row = 0; row < 5; row++)
    {
        for (i = 0; i < 4; i++)
        {
            /* 위치까지 맞으면 초록색, 숫자만 맞으면 노란색으로 표시합니다. */
            if (colors[i] == 1)
            {
                SetConsoleTextAttribute(
                    console,
                    FOREGROUND_GREEN | FOREGROUND_INTENSITY);
            }
            else if (colors[i] == 2)
            {
                SetConsoleTextAttribute(
                    console,
                    FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_INTENSITY);
            }
            else
            {
                SetConsoleTextAttribute(console, default_color);
            }

            print_digit_row(code[i] - '0', row);
            SetConsoleTextAttribute(console, default_color);

            if (i < 3)
                printf("  ");
        }

        printf("\n");
    }

    /* 다른 출력에 색상이 영향을 주지 않도록 기본 색상으로 복원합니다. */
    SetConsoleTextAttribute(console, default_color);
```

기능 추가(27) 디지털 비밀금고 - 색깔 힌트

```
int digit_colors[4] = { 0, 0, 0, 0 };
```

```
for (i = 0; i < 4; i++)
{
    matched[i] = 0;

    if (guess[i] == secret[i])
    {
        correct_position++;
        matched[i] = 1;
        digit_colors[i] = 1; /* 초록색: 숫자와 위치 모두 일치 */
    }
    else
    {
        is_correct = 0;
        digit_colors[i] = 0;
    }
}
```

```
/* 위치는 다르지만 숫자가 일치하는 경우를 계산합니다. */
for (i = 0; i < 4; i++)
{
    if (guess[i] != secret[i])
    {
        for (j = 0; j < 4; j++)
        {
            if (!matched[j] && guess[i] == secret[j])
            {
                correct_number++;
                matched[j] = 1;
                digit_colors[i] = 2; /* 노란색: 숫자만 일치 */
                break;
            }
        }
    }
}
```

기능 추가(27) 디지털 비밀금고 - 색깔 힌트

```
===== DIGITAL SECRET VAULT =====

네 자리 비밀번호를 맞혀 금고를 여세요!
비밀번호는 0부터 9까지의 숫자로 구성됩니다.
각 숫자는 중복되지 않습니다.

현재 금고 입력 번호

#######  ########  #######  ##    ##
     ##  ##    ##       ##  ##    ##
########  ##    ##       ##  ########
     ##  ##    ##       ##        ##
#######  ########       ##        ##

색상 힌트: 초록색 = 위치까지 일치, 노란색 = 숫자만 일치

시도 횟수: 3회

========== 이전 시도 기록 ==========
회차    입력 번호    위치 일치    숫자 일치
-------------------------------------
  1회      1234         1개         1개
  2회      3564         2개         0개
  3회      3074         2개         1개
=====================================

네 자리 번호 입력:
```

```
########  ########  ########  ##    ##
      ##        ##  ##    ##  ##    ##
########        ##  ########  ########
      ##        ##        ##        ##
########        ##        ##        ##

색상 힌트: 초록색 노란색
초록색: 숫자와 위치 모두 일치 / 노란색: 숫자만 일치

시도 횟수: 5회
숫자와 위치가 모두 일치하는 숫자: 4개
숫자만 일치하는 숫자: 0개
```

기능 추가(28) 디지털 비밀금고 - 6자리 중복 금고

```c
/* 디지털 비밀금고 시도 기록 구조체 */
struct VaultAttempt
{
    char guess[7];
    int correct_position;
    int correct_number;
};
void print_digital_vault_code(const char code[], const int colors[], int length);
```

```c
/* 8번 모드: 네 자리 또는 중복 가능한 여섯 자리 디지털 비밀금고입니다. */
void play_digital_vault_game(void)
{
    int vault_type;
    int ch;

    while (1)
    {
        system("cls");

        printf("===== DIGITAL SECRET VAULT =====\n\n");
        printf("입장할 비밀금고를 선택하세요.\n\n");
        printf("1. 일반 금고 - 4자리 / 숫자 중복 불가\n");
        printf("2. 확장 금고 - 6자리 / 숫자 중복 가능\n\n");
        printf("선택: ");

        if (scanf("%d", &vault_type) != 1)
        {
            printf("\n잘못된 입력입니다. 1 또는 2를 입력하세요.\n");
            while ((ch = getchar()) != '\n' && ch != EOF) {}
            wait_seconds(1.0);
            continue;
        }
```

기능 추가(28) 디지털 비밀금고 - 6자리 중복 금고

```
while ((ch = getchar()) != '\n' && ch != EOF) {}

if (vault_type == 1)
{
    play_digital_vault_mode(4, 0);
    return;
}
else if (vault_type == 2)
{
    play_digital_vault_mode(6, 1);
    return;
}

printf("\n잘못된 선택입니다. 1 또는 2를 입력하세요.\n");
wait_seconds(1.0);
```

```
if (allow_duplicates)
{
    /* 6자리 금고는 각 자리를 독립적으로 뽑아 숫자 중복을 허용합니다. */
    for (i = 0; i < code_length; i++)
        secret[i] = (char)('0' + rand() % 10);
}
else
{
    /* 4자리 금고는 0부터 9까지의 숫자를 섞어 중복 없이 선택합니다. */
    for (i = 0; i < 10; i++)
        digits[i] = i;

    /* Fisher-Yates 방식으로 숫자를 무작위로 섞습니다. */
    for (i = 9; i > 0; i--)
    {
        j = rand() % (i + 1);
        temp = digits[i];
        digits[i] = digits[j];
        digits[j] = temp;
    }

    for (i = 0; i < code_length; i++)
        secret[i] = (char)('0' + digits[i]);
}
```

기능 추가(28) 디지털 비밀금고 - 6자리 중복 금고

```
===== DIGITAL SECRET VAULT =====

입장할 비밀금고를 선택하세요.

1. 일반 금고 - 4자리 / 숫자 중복 불가
2. 확장 금고 - 6자리 / 숫자 중복 가능

선택: 2
```

```
===== DIGITAL SECRET VAULT =====

6자리 비밀번호를 맞혀 금고를 여세요!
비밀번호는 0부터 9까지의 숫자로 구성됩니다.
각 숫자는 중복될 수 있습니다.

현재 금고 입력 번호

########  ########  ##        ##        ##        ########
##        ##        ##        ##        ##        ##    ##
########  ########  ########  ########  ########  ########
      ##        ##  ##    ##  ##    ##  ##    ##  ##    ##
########  ########  ########  ########  ########  ########

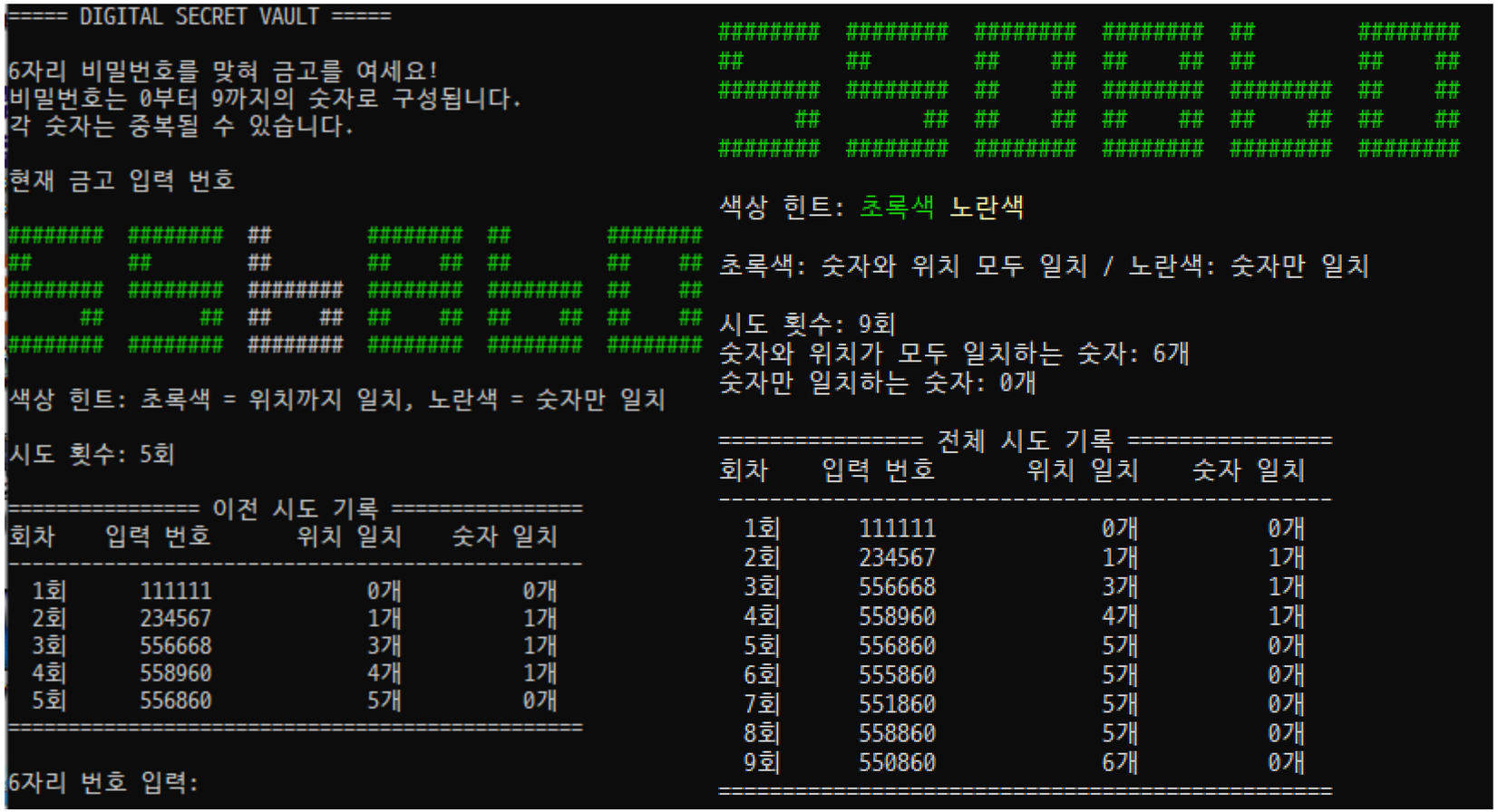
색상 힌트: 초록색 = 위치까지 일치, 노란색 = 숫자만 일치

시도 횟수: 3회

================ 이전 시도 기록 ================
회차    입력 번호       위치 일치    숫자 일치
------------------------------------------------
  1회      111111            0개          0개
  2회      234567            1개          1개
  3회      556668            3개          1개
================================================

6자리 번호 입력:
```

기능 추가(28) 디지털 비밀금고 - 6자리 중복 금고



기능 추가(29) 디지털 타이머

```
/* 9번 모드: 입력한 초부터 0초까지 디지털 숫자로 카운트다운합니다. */
void play_timer_mode(void)
{
    int total_seconds;
    int remaining_seconds;
    int hours;
    int minutes;
    int seconds;
    long time_number;
    int ch;
    HANDLE console = GetStdHandle(STD_OUTPUT_HANDLE);
    CONSOLE_SCREEN_BUFFER_INFO console_info;
    WORD default_color =
        FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_BLUE;
    WORD timer_color =
        FOREGROUND_GREEN | FOREGROUND_BLUE | FOREGROUND_INTENSITY;
    std::chrono::steady_clock::time_point start_time;

    /* 현재 콘솔의 기본 글자색을 저장합니다. */
    if (GetConsoleScreenBufferInfo(console, &console_info))
        default_color = console_info.wAttributes;

    system("cls");
    printf("===== 디지털 타이머 =====\n\n");
    printf("타이머 시간을 초 단위로 입력하세요. (1~359999초)\n");
    printf("입력 시간은 최대 99시간 59분 59초입니다.\n\n");
    printf("초 입력: ");
```

```
    if (scanf("%d", &total_seconds) != 1)
    {
        printf("잘못된 입력입니다. 숫자를 입력해 주세요.\n");
        while ((ch = getchar()) != '\n' && ch != EOF)
        {
        }
        printf("\n아무 키나 누르면 메인 메뉴로 돌아갑니다.\n");
        read_key_press();
        return;
    }
    while ((ch = getchar()) != '\n' && ch != EOF)
    {
    }

    if (total_seconds < 1 || total_seconds > 359999)
    {
        printf("\n1초 이상 359999초 이하로 입력해 주세요.\n");
        printf("\n아무 키나 누르면 메인 메뉴로 돌아갑니다.\n");
        read_key_press();
        return;
    }
```

기능 추가(29) 디지털 타이머

```
/* steady_clock을 사용하여 시스템 시각 변경과 관계없이 시간을 측정합니다. */
start_time = std::chrono::steady_clock::now();

for (remaining_seconds = total_seconds;
     remaining_seconds >= 0;
     remaining_seconds--)
{
    /* 남은 초를 시:분:초 형식으로 변환합니다. */
    hours = remaining_seconds / 3600;
    minutes = (remaining_seconds % 3600) / 60;
    seconds = remaining_seconds % 60;
    time_number = hours * 10000L + minutes * 100L + seconds;

    system("cls");
    printf("===== DIGITAL TIMER =====\n\n");
    printf("남은 시간\n\n");

    set_text_color(timer_color);
    print_digital_time(time_number);
    SetConsoleTextAttribute(console, default_color);

    if (remaining_seconds == 0)
        break;

    /* 매초 일정한 시점을 기준으로 다음 숫자를 표시합니다. */
    std::this_thread::sleep_until(
        start_time + std::chrono::seconds(total_seconds - remaining_seconds + 1));
}
```

```
system("cls");
printf("===== DIGITAL TIMER =====\n\n");
printf("타이머 종료!\n\n");

set_text_color(timer_color);
print_digital_time(0);
SetConsoleTextAttribute(console, default_color);

/* 카운트다운이 끝나면 종료 알림음을 세 번 재생합니다. */
Beep(880, 300);
Beep(988, 300);
Beep(1047, 600);

printf("\n타이머가 종료되었습니다.\n");
printf("\n아무 키나 누르면 메인 메뉴로 돌아갑니다.\n");
read_key_press();
```

기능 추가(29) 디지털 타이머

```
===== 디지털 타이머 =====

타이머 시간을 초 단위로 입력하세요. (1~359999초)
입력 시간은 최대 99시간 59분 59초입니다.

초 입력: 10
```

```
===== DIGITAL TIMER =====

남은 시간

######## ########     ########  ########        ##    ########
##    ## ##    ##  ##  ##    ##  ##    ##  ##    ##    ##    ##
##    ## ##    ##      ##    ##  ##    ##        ##    ##    ##
##    ## ##    ##  ##  ##    ##  ##    ##  ##    ##    ##    ##
######## ########     ########  ########        ##    ########
```

```
===== DIGITAL TIMER =====

타이머 종료!

######## ########     ########  ########     ########  ########
##    ## ##    ##  ##  ##    ##  ##    ##  ##  ##    ##  ##    ##
##    ## ##    ##      ##    ##  ##    ##      ##    ##  ##    ##
##    ## ##    ##  ##  ##    ##  ##    ##  ##  ##    ##  ##    ##
######## ########     ########  ########     ########  ########

타이머가 종료되었습니다.

아무 키나 누르면 메인 메뉴로 돌아갑니다.
```

기능 추가(30) 기본 글자색 설정

```c
/* 10번 모드: 디지털 숫자를 제외한 일반 글자색을 설정합니다. */
void play_text_color_mode(void)
{
    WORD colors[15];
    WORD background_color = original_console_color & 0xF0;
    const char *color_names[15] =
    {
        "파란색", "초록색", "청록색", "빨간색", "자주색",
        "노란색", "밝은 회색", "검은색", "밝은 파란색", "밝은 초록색",
        "밝은 청록색", "밝은 빨간색", "밝은 자주색", "밝은 노란색",
        "흰색"
    };
    int i;
    int choice;
    int ch;
    const char *current_color_name = "사용자 지정 색상";

    /* 기존 콘솔 배경색은 유지하고 글자색만 바꿉니다. */
    colors[0] = background_color | FOREGROUND_BLUE;
    colors[1] = background_color | FOREGROUND_GREEN;
    colors[2] = background_color | FOREGROUND_GREEN | FOREGROUND_BLUE;
    colors[3] = background_color | FOREGROUND_RED;
    colors[4] = background_color | FOREGROUND_RED | FOREGROUND_BLUE;
    colors[5] = background_color | FOREGROUND_RED | FOREGROUND_GREEN;
    colors[6] = background_color | FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_BLUE;
    colors[7] = background_color;
    colors[8] = background_color | FOREGROUND_BLUE | FOREGROUND_INTENSITY;
    colors[9] = background_color | FOREGROUND_GREEN | FOREGROUND_INTENSITY;
    colors[10] = background_color | FOREGROUND_GREEN | FOREGROUND_BLUE | FOREGROUND_INTENSITY;
    colors[11] = background_color | FOREGROUND_RED | FOREGROUND_INTENSITY;
    colors[12] = background_color | FOREGROUND_RED | FOREGROUND_BLUE | FOREGROUND_INTENSITY;
    colors[13] = background_color | FOREGROUND_RED | FOREGROUND_GREEN | FOREGROUND_INTENSITY;
    colors[14] = background_color | FOREGROUND_RED | FOREGROUND_GREEN |
                 FOREGROUND_BLUE | FOREGROUND_INTENSITY;
```

```c
for (i = 0; i < 15; i++)
{
    if (colors[i] == base_text_color)
    {
        current_color_name = color_names[i];
        break;
    }
}
```

기능 추가(30) 기본 글자색 설정

```
while (1)
{
    system("cls");
    set_text_color(base_text_color);

    printf("===== 기본 글자색 설정 =====\n\n");
    printf("디지털 숫자의 색상은 변경하지 않고, 일반 글자색만 변경합니다.\n\n");
    printf("현재 기본 글자색: %s\n\n", current_color_name);

    for (i = 0; i < 15; i++)
    {
        set_text_color(colors[i]);
        printf("%2d. %s\n", i + 1, color_names[i]);
    }

    set_text_color(base_text_color);
    printf("\n 0. 이전 메뉴로 돌아가기\n\n");
    printf("변경할 글자색을 선택하세요: ");

    if (scanf("%d", &choice) != 1)
    {
        printf("잘못된 입력입니다. 숫자를 입력해 주세요.\n");
        while ((ch = getchar()) != '\n' && ch != EOF)
        {
        }
        wait_seconds(1.0);
        continue;
    }
```

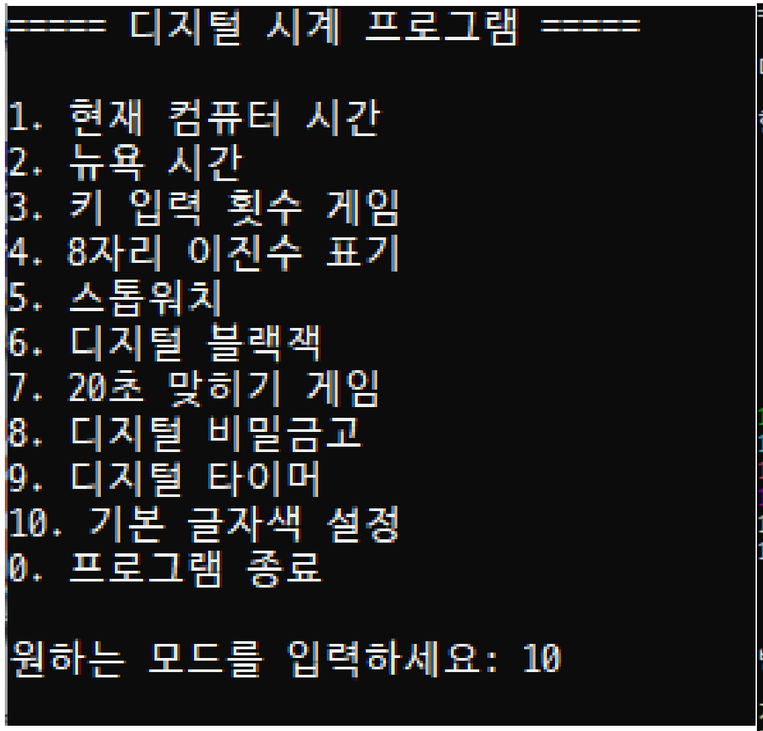
```
while ((ch = getchar()) != '\n' && ch != EOF)
{
}

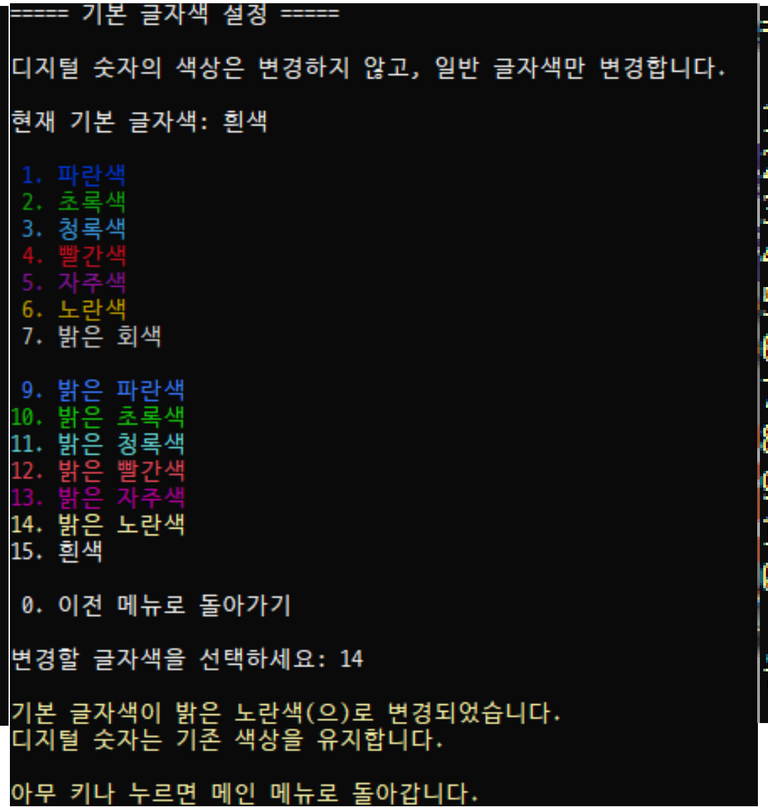
if (choice == 0)
    return;

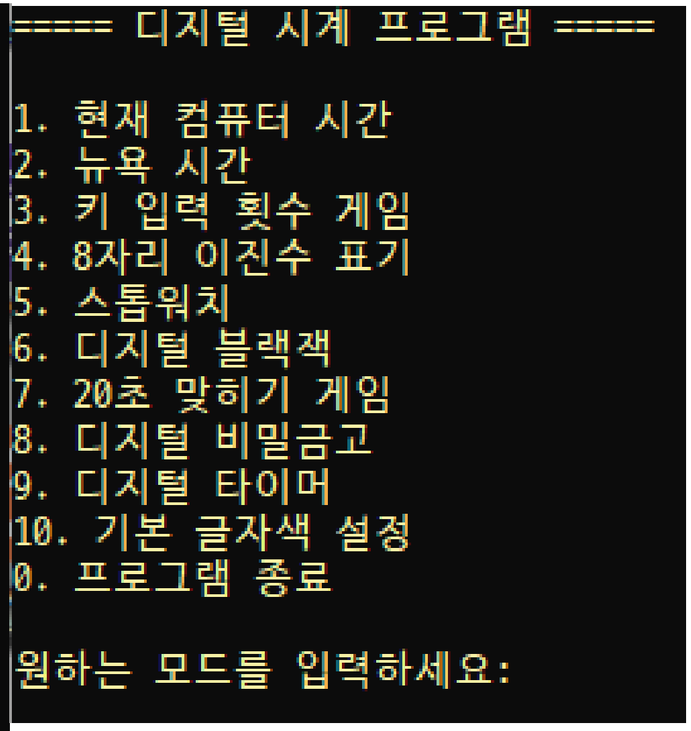
if (choice < 1 || choice > 15)
{
    printf("잘못된 선택입니다. 0~15 사이의 번호를 입력해 주세요.\n");
    wait_seconds(1.0);
    continue;
}

base_text_color = colors[choice - 1];
set_text_color(base_text_color);
printf("\n기본 글자색이 %s(으)로 변경되었습니다.\n", color_names[choice - 1]);
printf("디지털 숫자는 기존 색상을 유지합니다.\n");
printf("\n아무 키나 누르면 메인 메뉴로 돌아갑니다.\n");
read_key_press();
return;
```

기능 추가(30) 기본 글자색 설정







점수

30/30점

```
2511        printf("\n타이머가 종료되었습니다.\n");
2512        printf("\n아무 키나 누르면 메인 메뉴로 돌아갑니다.\n");
2513        read_key_press();
2514    }
```

감사합니다

2021963020

작성자 : 김재홍